The Journal of Supercomputing (2025) 81:950
https://doi.org/10.1007/s11227-025-07414-4

Optimizing SBERT for long text clustering: two novel approaches with empirical insights

Yasin Ortakci[1] · **Burak Borhan**[1]

Accepted: 6 May 2025

Abstract

Transformer-based Large Language Models (LLMs), which have recently gained popularity, have significantly impacted the various fields of natural language processing. One of them is text clustering, which involves categorizing the huge texts produced by today's digital world into meaningful groups. LLMs enable text clustering with a more semantic and contextualized approach than traditional methods. One such model is Sentence-BERT (SBERT), which has been modified to detect semantic similarity between texts. Before being clustered, texts need to be transformed into numerical text embeddings. SBERT-based models have shown promise in generating meaningful sentence embeddings. However, they face limitations when dealing with long texts that exceed their maximum token limit. In this context, this study proposes two distinct methods to overcome these limitations and enhance the performance of SBERT models for clustering long text. The proposed methods are combined with various SBERT models, and their combinations are compared to the existing default method on three datasets containing lengthy texts. This study evaluates the impact of these methods on the models and their contributions to clustering performance. The findings indicate that the proposed methods exhibit a higher clustering performance of up to 14% than the default method in text clustering. Additionally, this study provides valuable insights into the text clustering performance of SBERT models, offering practical implications for further research and applications.

Keywords Text clustering · Long text · Sentence-BERT · Sentence embeddings · Transformers · Semantic similarity

✉ Yasin Ortakci
yasinortakci@karabuk.edu.tr

Burak Borhan
2328126507@ogrenci.karabuk.edu.tr

[1] Department of Computer Engineering, Karabuk University, Demir Çelik Kampüsü, 78050 Merkez, Karabük, Turkey

Published online: 02 June 2025

1 Introduction

Text clustering groups unlabeled text data based on similarities, aiming to maximize semantic similarity within groups and dissimilarity between groups. It plays a vital role in various fields, such as data organization [1], topic modeling [2], information retrieval [3], recommendation systems [4], document summarization [5], and social media analysis [6]. Since clustering methods cannot process text directly, one of the initial stages of text clustering is transforming the text into numerical vector representations. Recently, transformer-based Large Language Models (LLMs) have emerged as one of the most sophisticated techniques for generating vector representations of texts. These advanced models extract deeper semantic meanings and comprehend contextual relationships within the text. They demonstrate enhanced performance across various Natural Language Processing (NLP) tasks, surpassing the capabilities of the previously employed methods [7, 8].

Bidirectional Encoder Representations from Transformers (BERT), one of the most significant LLMs, revolutionized NLP research by introducing bidirectional context modeling and attention mechanisms. This marked a paradigm shift, allowing semantic and contextual relationships to be captured more effectively. However, while models like BERT are primarily designed for natural language understanding, they are not optimized for unsupervised tasks such as text clustering [9]. Since BERT generates word-level embeddings, it requires significant computational resources for sentence-level processing. To address this limitation, Sentence-BERT (SBERT), a modified version of BERT specifically fine-tuned to produce dense sentence embeddings suitable for semantic similarity tasks, was introduced. This modification makes SBERT particularly effective for unsupervised tasks such as text clustering, where semantic similarity is paramount. SBERT uses a Siamese network architecture to generate fixed-size sentence-level embeddings and significantly outperforms BERT and other existing methods in producing meaningful sentence representations.

Reviewing the existing literature on implementing LLMs in text clustering, we realized that although some studies incorporate BERT word embeddings, more comprehensive research on SBERT-based models is required within this context. In light of this observation, this study focuses on the text clustering performance of SBERT-based generic models, particularly clustering long text. When generating sentence embeddings, SBERT models are constrained by a maximum token limit, typically ranging from 75 to 512 tokens, depending on the model training. If the word counts of the texts exceed this limit, the excess content is truncated, resulting in incomplete text representation and low clustering performance [10]. This poses a challenge for real-world documents, such as news articles, legal documents, or user reviews, which often span thousands of tokens. To overcome these problems, this paper introduces two different approaches, Sentence-Level (SL) and Block-Level (BL) SBERT embedding generation methods. The SL generates sentence embeddings by splitting long texts into individual sentences, while the BL produces sentence embeddings by dividing long texts into

chunks that contain tokens up to the model's maximum limit. Then, both methods aggregate these sentence embedding units to create a general text embedding. Unlike default SBERT models, which truncate the text and discard excess, these approaches preserve the full semantic content of the input, offering a scalable and efficient solution for clustering long documents regardless of the token number constraint. In this context, the proposed methods are integrated with five distinct SBERT generic pre-trained models (BERT, RoBERTa, ALBERT, DistilBERT, MPNET), and their compatibility is thoroughly analyzed. To evaluate clustering performance, two algorithms, K-means and K-medoids, are applied to the sentence embeddings generated by these integrations.

These approaches are rigorously tested on three diverse datasets (Amazon Reviews, BBC News, and Fake-Real News) to assess performance across various text types and provide robust insights into their compatibility. Experimental results show that both methods generally outperformed the default method in many cases. SL also produces better results than BL, achieving up to 14% improvement in the clustering accuracy with K-means. On the other hand, the clustering performance of the proposed method was comparable to state-of-the-art models such as LongFormer and BigBird, demonstrating competitive results. Notably, BERT and DistilBERT exhibited the highest compatibility with the proposed methods, producing more distinct clusters compared to the default approach. These findings underscore the potential of SL and BL to enhance SBERT-based text clustering, particularly for long texts, and provide valuable insights for selecting and applying generic SBERT models in unsupervised NLP tasks. This study used generic models without fine-tuning since finding appropriate training data to fine-tune the models in clustering is often challenging. Consequently, the findings also provide a basis for evaluating the transfer learning capabilities of generic SBERT models in text clustering. The main contributions of this paper can be summarized as follows:

- An extensive related work regarding text representations used in clustering is presented.
- Two novel methods are proposed for long text clustering, eliminating the maximum token limit and improving performance. The compatibility of these methods with different SBERT models is demonstrated.
- A comparative analysis of SBERT generic models' performance in text clustering, which offers valuable insights to researchers on model selection, is presented.

The paper is structured as follows: Sect. 2 reviews related work on text representation methods for clustering, ranging from traditional approaches to contextual embeddings. Section 3 describes the generation of text embeddings with LLMs and how SBERT models produce sentence embeddings. Section 4 describes the proposed SL and BL methods, the data pre-processing, and the K-means and K-medoids clustering methods. Section 5 presents experimental results on three datasets, demonstrating the clustering performance of SL and BL over the default method and benchmark transformers developed for long text processing. Section 6 provides a detailed discussion of the proposed approaches, offering insights into

their performance, model compatibility, and broader implications for text clustering. Section 7 addresses the limitations of the proposed methods and outlines future research directions. Finally, Sect. 8 concludes with general findings on model compatibility and broader implications for SBERT-based text clustering.

2 Related work

The numerical representation of texts heavily influences the performance of text clustering. In the existing literature, studies on creating these numerical representations can be categorized into three main groups: traditional approaches, content-based word embeddings, and contextualized word embeddings [11]. In this regard, this section presents a comprehensive review of diverse text clustering studies based on their text representation methods.

2.1 Traditional approaches

Traditional approaches generally rely on analyzing word frequencies and rarity within a text corpus. A common method is Bag-of-Words (BoW), which calculates word distributions to create basic vector representations. However, BoW's focus on word distribution, disregarding word order, limits its performance. To address this, some studies have integrated BoW with the co-occurrence feature or combined it with Word2 Vec to improve clustering outcomes [12, 13].

Another traditional method, Term Frequency-Inverse Document Frequency (TF-IDF), generates word representations based on their importance by weighing a word's frequency in a document against its rarity across all documents. TF-IDF has been applied in tasks such as clustering news articles and social media users in two separate studies [14, 15], in which the authors recommend exploring more advanced text vectorization techniques due to TF-IDF's limited capacity for semantic understanding. To achieve better clustering results, some studies have combined TF-IDF with Word2 Vec [16, 17] or used dimension reduction techniques to deal with TF-IDF's large, sparse vector representations [18]. Overall, traditional methods produce text representations relying heavily on word frequencies, resulting in sparse matrices that neglect the semantic nuances of the text [19].

2.2 Content-based word embedding methods

These are neural network-based methods, trained on large corpora, that capture the semantic relationships between words by generating dense, fixed-size, static embedding vectors. These vectors position similar words closely in the vector space. One prominent method, Word2 Vec [20], is notable for the fact that when simple vectorial operations combine its word embedding, the resulting vector captures the meaning of the combined words [21]. For example, the " king − man + woman" operation yields a vector close to "queen". Word2 Vec has produced semantically meaningful embeddings in text clustering studies [22, 23]. Another approach, GloVe

[24], uses word co-occurrence statistics to create embeddings, improving text representation for clustering in various studies [25, 26]. Given that Word2 Vec and Glove suffer from generating word embeddings from out-of-vocabulary words, Facebook has introduced FastText [27], which uses character n-grams in word embedding production, yielding promising results in [28, 29].

Compared to traditional approaches such as BoW and TF-IDF, these methods offer significant advantages for text clustering. Unlike sparse, high-dimensional representations that ignore word meaning, content-based embeddings provide compact vectors that encode semantic similarity, enabling more effective clustering of texts with related meanings. However, these methods generate context-free embeddings and struggle with polysemy and contextualization. For instance, semantically opposite sentences like "I am going home" and " I am not going home" may be represented closely in the vector space [30].

2.3 Contextualized word embedding methods

Unlike neural network-based models, contextualized word embedding models generate word embeddings that adapt to the surrounding content. Early examples include ULMFiT [31] and ELMo [32], which are modifications of the LSTM. With the introduction of the self-attention mechanism [33], the focus of research shifted to transformer-based LLMs such as GPT [34] and BERT [35]. These models capture deeper linguistic features related to syntax, semantics, and context [36]. Although there are some studies exploring GPT-based word embeddings for semantic search [37], GPT models, primarily designed for language generation, are less suitable for text similarity tasks [30]. Similarly, [38–42] use BERT for text clustering and do not generate text embeddings of the whole text, but compute them by averaging individual word embeddings. Since these averages are often worse than the Glove averages, a new model, the SBERT, has been introduced specifically for measuring the semantic similarity of text. SBERT represents texts by considering the whole text rather than word by word, ensuring that similar meaningful texts have similar sentence embeddings. SBERT has outperformed widely used methods such as InferSent and Universal Sentence Encoder in semantic text similarity tasks [43].

Unlike sparse, frequency-based representations like BoW and TF-IDF, which lack semantic depth, or context-free embeddings such as Word2 Vec, GloVe, and FastText, which fail to capture shifts in word meaning across different contexts, contextualized embeddings dynamically adapt to context, effectively capturing nuanced syntactic, semantic, and pragmatic features. This makes them more accurate at clustering complex or ambiguous text by capturing each word's specific role in a sentence or document.

2.4 Current status of text clustering with SBERT

SBERT's success in semantic similarity tasks and its ability to generate robust, contextually informed sentence embeddings have made it a good option in recent text clustering studies. Accordingly, several studies [44–46] have transformed texts into

sentence embeddings using SBERT models, applying various clustering algorithms to perform text clustering. Similarly, [47] investigated the impact of different pooling techniques on SBERT's clustering performance, while [48] proposed a consensus clustering approach by aggregating results from multiple SBERT models. Furthermore, studies comparing SBERT-generated text embeddings with other word embedding methods [49, 50] have demonstrated superior clustering performance with SBERT.

SBERT models are pre-trained and constrained by a maximum token limit during training, which poses challenges for generating embeddings for long texts exceeding this limit. Despite this limitation, [51] and [52] explored text clustering on long documents using default SBERT models, ignoring the maximum token constraint. To address SBERT's limitations in embedding long documents, [53]generated individual sentence embeddings, grouped similar sentences, and derived a document-level embedding by selecting a representative sentence from each group. Our study adopts a similar approach by extracting sentence embeddings for all sentences in a text, but instead of relying on representative sentences, it integrates all sentence embeddings to produce a holistic document embedding. Additionally, [54] and [55] employed fine-tuned SBERT models for text clustering. However, since suitable training data for fine-tuning is not always available, our study evaluates the general-purpose clustering performance of generic SBERT models without fine-tuning.

Recent advancements in transformer-based models, such as Longformer [56] and BigBird [57], designed for long documents using specialized attention mechanisms for various NLP tasks, also provide relevant benchmarks. In this work, we conducted a performance analysis by comparing our results with those of these benchmark models. Although these models are designed for long text processing, they are also constrained by a 4096-token limit. In contrast, our model is not restricted by any token length limitations, enabling more flexible and scalable clustering of long texts. Ultimately, our study contributes to the literature by proposing a novel approach to clustering long texts.

3 Text embeddings

The quality of text clustering heavily depends on generating text embeddings, generally called sentence embeddings. This chapter introduces LLMs that serve as the source of SBERT versions used in this study to extract sentence embeddings. As BERT is a breakthrough in LLMs, it briefly explains BERT. Subsequently, it highlights the differences between other models with a similar structure to BERT.

3.1 Embeddings with BERT-based models

BERT [35] is a milestone LLM for detecting semantic and logical relationships between texts, released by Google in late 2018. It is an unsupervised pre-trained model that leverages massive corpora such as Wikipedia and BooksCorpus [58] for the training. During training, BERT employs specific tasks: Masked Language

Model (MLM) and Next Sentence Prediction (NSP). In the MLM task, random tokens within the text are masked, and the model aims to predict these masked tokens. The NSP task aims to determine the relationship between two consecutive sentences. A distinguishing feature of BERT from other transformer-based models is its ability to simultaneously model the left and right contexts of the text bi-directionally during the pre-training phase. This unique feature allows BERT to capture contextual information and produce text embeddings more effectively. Although the pre-trained BERT model is usually fine-tuned for various downstream tasks, it can also be applied to different NLP tasks with a feature-based approach [59, 60].

BERT utilizes only the encoder block of transformers as it performs unsupervised learning. The network of BERT consists of multiple encoders that receive the output of the previous blocks as their input (Fig. 1). Each encoder incorporates a multi-head self-attention mechanism and a feed-forward neural network module. The self-attention mechanism enables the detection of word relationships within the text. At the same time, the feed-forward neural network enhances the semantic representation of words by mapping them to a higher-dimensional space. The base model of BERT consists of 12 encoder layers, 12 attention heads, a hidden dimension of 768, and approximately 110 million parameters. To process the input text, BERT tokenizes it using the WordPiece method [61] and combines the resulting tokenized vectors with segment embeddings and positional coding vectors. These combined vectors are then processed through the encoder blocks. As output, BERT generates a classification label [CLS] of the text and word embedding vectors of the individual words, each represented in 768 dimensions.

RoBERTa [62], another LLM used in this study, is a modification of BERT that shares the same transformer architecture. However, it introduces several differences in its training approach. These include training with a larger dataset over an extended period, eliminating the NSP task, and implementing an alternative dynamic masking model to MLM. Moreover, RoBERTa utilizes byte pair encoding [63] for word tokenization instead of the WordPiece method [64].

ALBERT (A Lite BERT) [65] is a compact version of BERT designed to decrease memory usage and training time. ALBERT scales the pre-trained BERT model by reducing the number of parameters without significant performance loss. Additionally, to enhance its performance, ALBERT employs sentence order prediction instead of the NSP task utilized in BERT.

DistilBERT [66], a distilled variant of BERT, achieves a 40% reduction in size compared to BERT using distillation techniques during pre-training. Despite this reduction, it maintains 97% of the language understanding capacity. Moreover, DistilBERT exhibits a 60% acceleration in model performance, enabling faster inference times.

MPNET [67] differs from BERT's masking approach. Instead of relying solely on MLM, MPNET incorporates both masked and permuted language modeling. This modified approach considers positional information and token dependencies when predicting word embeddings through masking. In addition, MPNET does not use the NSP task of BERT.

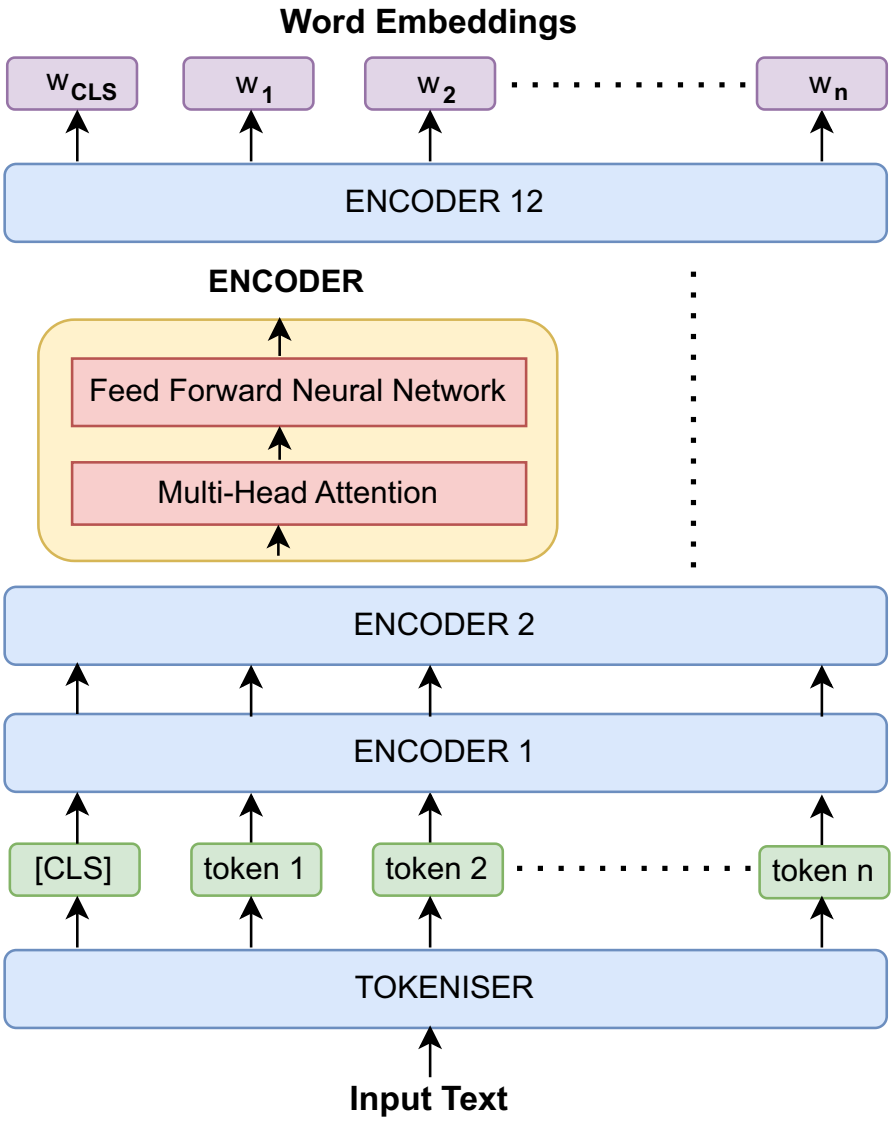

Fig. 1 Basic BERT architecture

3.2 Embeddings with sentence transformer

Although BERT produces successful results in text pair regression, it suffers from a significant disadvantage. In text similarity, BERT requires processing both texts together using a cross-coder, which incurs high computational cost and time loss. This structural limitation, which prevents BERT from creating independent sentence embeddings, renders it unsuitable for unsupervised tasks such as clustering.

Researchers have explored alternative approaches to overcome this limitation, such as averaging word embeddings from BERT's final layer or using the CLS token. However, these methods have generally yielded lower results compared to the average of the Glove embeddings.

Reimers and Gurevych introduced SBERT [43], a specialized modification of BERT designed to measure the semantic similarity of texts. SBERT generates unified sentence embeddings for the input text by employing either the CLS token, average pooling, or maximum pooling method. These methods process the word embeddings produced by the LLMs for each word, resulting in a single, fixed-size sentence representation. SBERT then fine-tunes the weights of these embeddings in a Siamese network to ensure that semantically similar sentences are close to each other in the vector space. SBERT employs cosine similarity as the regression objective function in fine-tuning, as depicted in Fig. 2. Consequently, it creates independent sentence embeddings suitable for clustering and semantic search. Even though it was not initially designed for transfer learning, SBERT has shown that it can be used in text clustering as a base model by producing more realistic sentence embeddings than existing state-of-the-art methods [68, 69]. SBERT models are also called sentence transformers.

4 Methodology

Based on the SBERT approach, many LLMs have been trained with different corpora to create distinct versions of sentence transformers. This study utilizes generic pre-trained sentence transformer models derived from BERT, RoBERTa, ALBERT, DistilBERT, and MPNET. Figure 3 displays the general overview of the applied methodology. After pre-processing the datasets, each model generates the sentence embeddings with the proposed methods. These embeddings are then normalized and

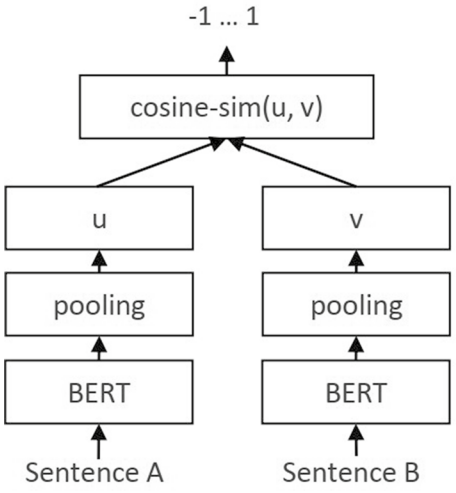

Fig. 2 Siamese network of SBERT [43]

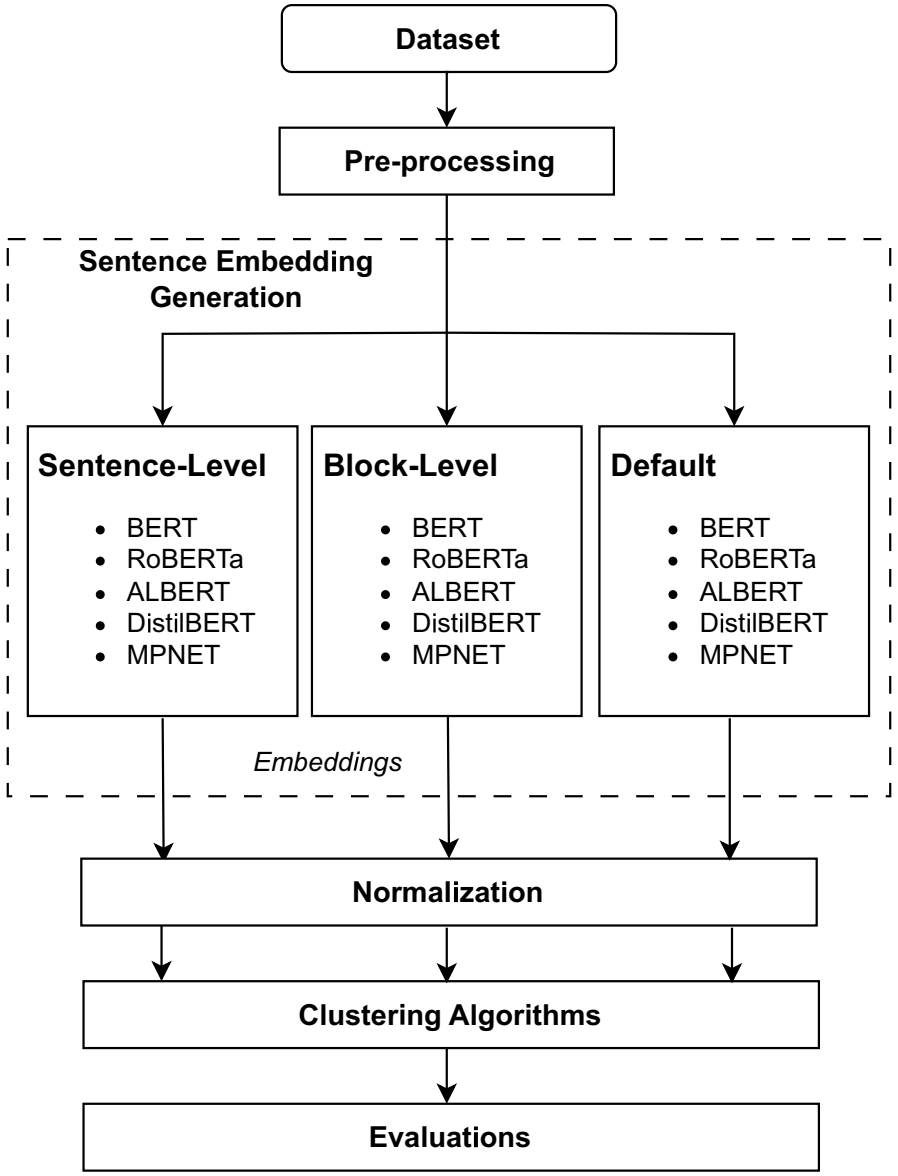

Fig. 3 The overview of the methodology

Table 1 Details of sentence transformer models

Base model	Sentence transformer model name	Max-seq-length
BERT	Bert-base-nli-mean-tokens	128
RoBERTa	Nli-roberta-base-v2	75
DistilBERT	Distilbert-base-nli-mean-tokens	128
ALBERT	Paraphrase-albert-small-v2	100
MPNET	All-mpnet-base-v2	128

subjected to K-means and K-medoids clustering. Finally, the clustering performance of each model with the proposed methods is evaluated.

4.1 Data pre-processing

Data pre-processing removes all non-alphabetic characters from the text in the dataset, except sentence separators such as ".", "?", "!". It also eliminates consecutive repetitions of these sentence separators and reduces them to a single character. In addition, it removes all URL addresses from the text. Considering the paramount role of contextualization within LLMs, this study deliberately chose not to remove stop words from the text, recognizing that these common linguistic elements can play a significant role in capturing contextual nuances and generating more accurate and contextually rich sentence embeddings.

4.2 Sentence embeddings generation

Table 1 lists the sentence transformers used in this study with their base LLMs. All the models employed in this study generate 768-dimensional sentence embeddings for each input text. These embeddings are sourced from the Hugging Face platform,[1] which offers a wealth of machine-learning resources in NLP.

Table 1 also shows the maximum number of tokens that each sentence transformer model can handle (denoted as *max_seq_length*) in an input text. During pre-training, these models are fine-tuned on the initial part of the input text that contains as many tokens as the *max_seq_length* parameter. While sentence transformers are theoretically capable of generating sentence embeddings for up to 512 tokens in a text, it's important to note that if a text exceeds the specified *max_seq_length* limit, the resulting embeddings become meaningless, as the model is not trained to generate embeddings for the excess portion. For instance, the SBERT model in Table 1 will be inadequate for generating embeddings of texts containing more than 128 tokens. When dealing with texts that go beyond this limit, the sentence embeddings become incomplete, leading to suboptimal performance in clustering longer texts.

To address this issue, this study introduced two innovative approaches: Sentence Level and Block Level. Both methodologies enable sentence embedding

[1] https://huggingface.co/sentence-transformers.

generation that is independent of text length, while also presenting a novel strategy for clustering long-form texts. On the other hand, in the subsequent sections of the paper, we will use the term "Default" method to refer to the traditional method, which is limited to generating sentence embeddings for a number of tokens equal to the *max_seq_length* in the text.

4.2.1 Sentence-level (SL)

The SL method addresses the token limitation by splitting input text into individual sentences, each processed independently to generate embeddings. To obtain the overall embeddings of the text, it averages all sentence embeddings as shown in Fig. 4a. This technique ensures that no sentence exceeds the model's *max_seq_length*, as sentences typically contain fewer tokens than the model's limit.

For a text with n sentences, each sentence S_i (where i ranges from 1 to n) is converted into a 768-dimensional embedding vector $\vec{s}_i$. As all the sentence transformers used in this study convert sentences into 768-dimensional vectors, each $\vec{s}_i$ represents the embedding vector of the ith sentence, with 768 components.

The sentence embedding representation of the whole text has the form:

$$\vec{E} = \begin{bmatrix} S_1 \\ \vdots \\ S_i \\ \vdots \\ S_n \end{bmatrix} = \begin{bmatrix} s_{1,1} & \cdots & s_{1,768} \\ \vdots & \ddots & \vdots \\ s_{i,1} & \cdots & s_{i,768} \\ \vdots & \ddots & \vdots \\ s_{n,1} & \cdots & s_{n,768} \end{bmatrix}$$

The overall text embedding, $\vec{E}$, is computed as the arithmetic mean of these sentence embeddings (S_i), as shown in Eq. (1):

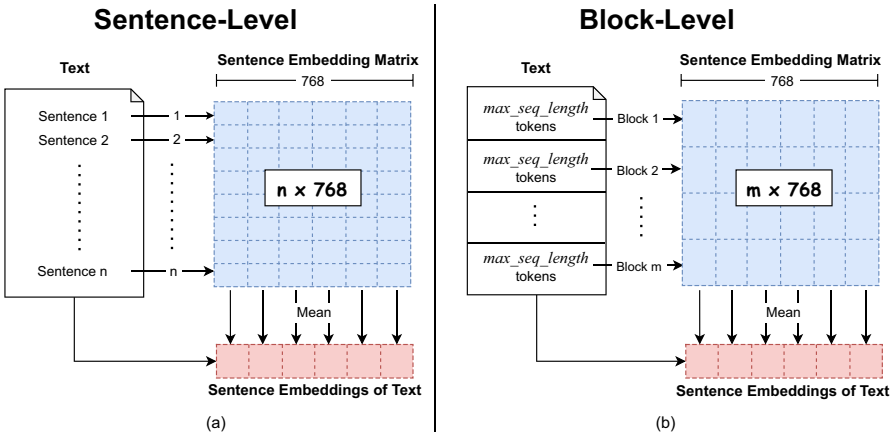

Fig. 4 The proposed sentence embeddings methods, **a** Sentence level sentence embeddings, **b** Block level sentence embeddings

$$E = \frac{1}{n}\sum_{i=1}^{n} S_i \tag{1}$$

where $\vec{E} = \begin{bmatrix} E_1 & E_2 & \dots & E_{768} \end{bmatrix}$ is a 768-dimensional vector, and $S_i = \begin{bmatrix} s_{i,1} & s_{i,2} & \dots & s_{i,768} \end{bmatrix}$ represents the embedding of the ith sentence. Each component of E, E_j (for $j=1,2,\dots,768$), is computed as in Eq. (2):

$$E_j = \frac{1}{n}\sum_{i=1}^{n} s_{i,j} \tag{2}$$

In Eq. (2), the summation $\sum_{i=1}^{n} s_{i,j}$ aggregates the jth components of the n sentence embeddings, and division by n results the average for the jth dimension. This process is applied across all 768 dimensions of E. The averaging preserves the semantic content of the text while mitigating the influence of varying sentence lengths. SL method excels in capturing fine-grained semantic nuances, making it particularly effective for texts with distinct sentence structures.

4.2.2 Block-level (BL)

The BL method divides the text into contiguous blocks, each containing up to *max_seq_length* tokens, constrained by the corresponding model. For a text which is divided into m blocks, each block B_i (where i ranges from 1 to m) is processed to produce a 768-dimensional embedding vector $\vec{b}_i$. Each $\vec{b}_i$ represents the embedding vector of the ith block, with 768 components. The embedding representation of the whole text has the form:

$$\vec{F} = \begin{bmatrix} B_1 \\ \vdots \\ B_i \\ \vdots \\ B_m \end{bmatrix} = \begin{bmatrix} b_{1,1} & \cdots & b_{1,768} \\ \vdots & \ddots & \vdots \\ b_{i,1} & \dots & b_{i,768} \\ \vdots & \ddots & \vdots \\ b_{m,1} & \dots & b_{m,768} \end{bmatrix}$$

The overall text embedding in the BL, denoted $\vec{F}$, is computed as the arithmetic mean of these block embeddings (B_i), as shown in Eq. (3):

$$F = \frac{1}{m}\sum_{i=1}^{m} B_i \tag{3}$$

where $\vec{F} = \begin{bmatrix} F_1 & F_2 & \dots & F_{768} \end{bmatrix}$ is a 768-dimensional vector, and $B_i = \begin{bmatrix} b_{1,1} & b_{j,2} & \dots & b_{1,768} \end{bmatrix}$ represents the embedding of the ith block. Each component of F, F_j (for $j=1, 2,\dots,768$), is computed as in Eq. (4):

$$F_j = \frac{1}{m}\sum_{i=1}^{n} b_{i,j} \tag{4}$$

In Eq. (4), the summation $\sum_{i=1}^{m} b_{i,j}$ aggregates the jth components of m block embeddings, and division by m computes the average. This process is applied across

all 768 dimensions of F. Unlike the SL method, BL maintains contextual coherence within larger text chunks, making it suitable for texts where semantic meaning spans multiple sentences. The block size is dynamically adjusted to the model's *max_seq_length*, ensuring no information is truncated.

Both SL and BL methods innovate by decoupling embedding generation from text length, enabling robust clustering of long texts. The SL method emphasizes sentence-level semantics, while BL prioritizes broader contextual patterns, offering complementary approaches to text representation.

4.3 Normalization

Feature normalization is a critical step in adapting sentence embeddings to the clustering, as it prevents inconsistencies and reduces the impact of outlier values. This process maps embeddings to the unit hypersphere, facilitating fair comparisons during clustering by mitigating magnitude disparities.

This study employed the L2-norm normalization method presented in Eq. (5).

$$e_i = \frac{e_i}{\|e\|} \tag{5}$$

$$\|e\| = \sqrt{\sum_{i=1}^{d} \left(e_i\right)^2}$$

where d=*768* and $\vec{E} = [e_1, e_2, ..., e_d]$ is the sentence embeddings vector

4.4 Text clustering

Before clustering, to reduce computational complexity and enhance clustering performance, the 768-dimensional normalized embeddings were projected into a 5-dimensional space using Uniform Manifold Approximation and Projection (UMAP[2]), a non-linear dimensionality reduction technique that preserves both local and global data structures. With extensive experiments, 5-dimension was determined as the optimal dimensionality for this reduction. Then, we employed K-means [70] and K-medoids [71] separately to cluster the text embeddings, which are partition-based, fast, scalable, and accurate clustering algorithms.

Let's $T = t_1, t_2, \ldots, t_n$ is a corpus consisting of n text, where each text (t_i) is represented by its sentence embedding vector $\vec{E} = [e_1, e_2, \ldots, e_{768}]$. K-Means iteratively scatters the texts into a set of k clusters, $C = C_1, C_2, \ldots, C_k$ aiming to minimize the objective function in Eq. (6). At each iteration, it assigns the texts to the closest cluster (C_j) by calculating the Euclidean distance between the text and cluster center and then computes the new cluster center as in Eq. (7).

[2] https://umap-learn.readthedocs.io/en/latest/basic_usage.html.

$$J = \sum_{j=1}^{k} \sum_{t_i \epsilon C_j} \| t_i - o_j \|^2 \tag{6}$$

where o_j is the center of C_j

$$o_j = \frac{\sum_{t_i \epsilon C_j}^{|C_j|} t_i}{| C_j |} \tag{7}$$

$|C_j|$ =number of text in C_j

K-medoids groups a dataset into k cluster by selecting cluster centers among the actual data points and call these centers as medoids, which minimize the total distance between points and their assigned medoid. In this study, K-medoids starts with k randomly selected text t_k as medoids, assigns each text t_i to the nearest medoid, and iteratively swaps medoids with other texts to reduce the total cost (C), which is sum of distances between t_k and t_i as in Eq. (8). For the distance calculation, the Euclidean metric is used in this study.

$$C = \sum_{t_k} \sum_{t_i \in T_i} \|t_i - t_k\|^2 \tag{8}$$

5 Experimental studies

This section delves into the performance evaluation of the proposed methods: SL and BL, by comparing the results obtained with those of the Default method. In this context, it presents a comprehensive analysis of the compatibility of sentence transformer models with the proposed methods. The evaluation metrics used in the analyses and the details of the datasets containing long texts are described below. The experiments were conducted on a laptop running the Microsoft Windows 11 Pro operating system, equipped with an Intel(R) Core(TM) i7-10750 H CPU @ 2.60GHz processor, 32 GB of RAM, and a 1 TB SSD hard drive. All coding in this study was developed using the Python programming language.

5.1 Datasets

In this study, a "long text" is defined as any text exceeding the *max_seq_length* constraint of sentence transformer models, as specified in Table 1. This section describes the Amazon Reviews, BBC News, and Fake-Real News datasets, which typically contain long texts. Sections 5.1.1–5.1.3 provide dataset details, while Sect. 5.1.4 presents qualitative and quantitative analyses, with key statistics summarized in Table 2. Histograms (Figs. 5, 6 and 7) illustrate the token count distributions, highlighting the prevalence of long texts across all datasets.

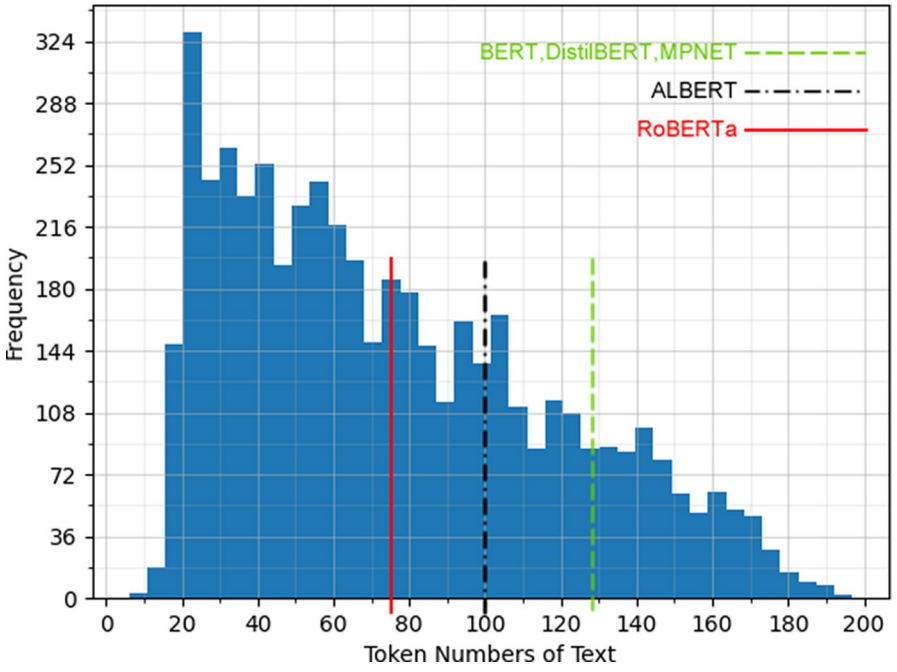

Fig. 5 Histogram of Amazon reviews dataset

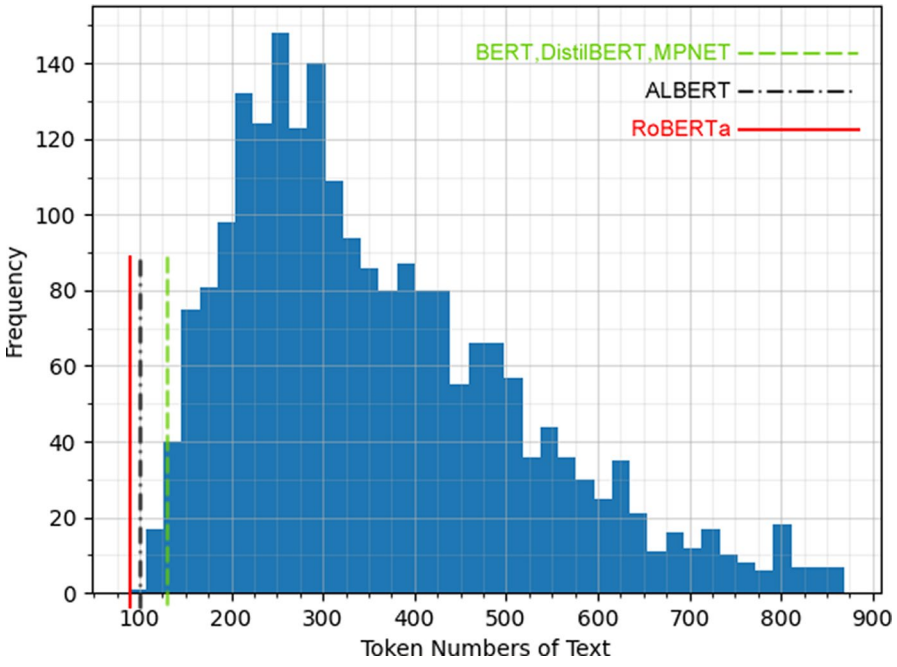

Fig. 6 Histogram of BBC news dataset

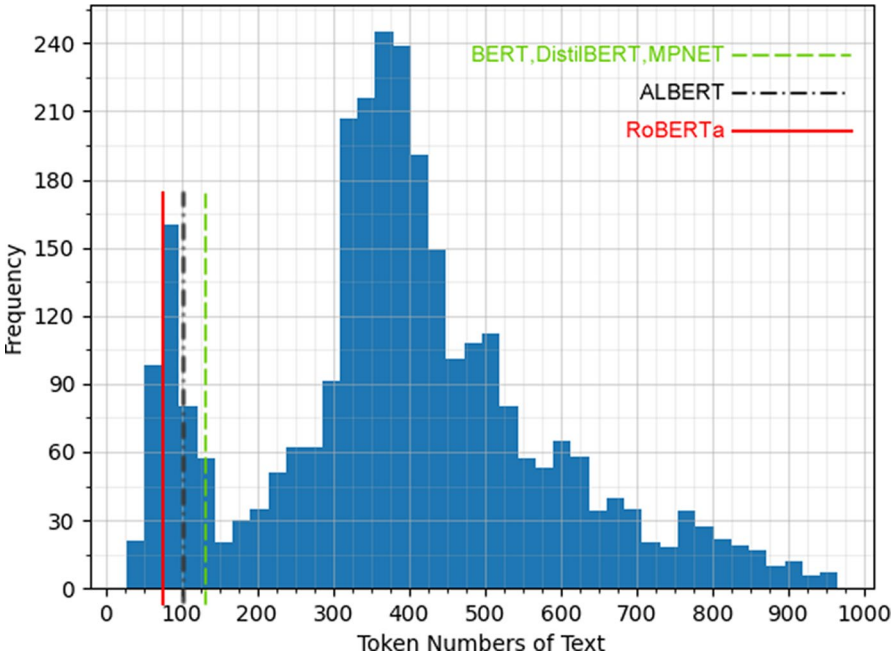

Fig. 7 Histogram of Fake–Real news dataset

Table 2 Statistics regarding the datasets (B: Business, P: Politics, S: Sport, T: Technology, E: Entertainment)

Metric	Amazon reviews	BBC News	Fake–real news
Number of records	5000	2225	3000
Categories	5 (Score 1–5)	5 (B,P,S,T,E)	2 (Real, Fake)
Mean token count	82	402	423
Exceeding RoBERTa	2382 (47.6%)	2225 (100%)	2883 (96.1%)
Exceeding ALBERT	1606 (32.1%)	2224 (99.9%)	2722 (90.7%)
Exceeding BERT/DistilBERT/MPNET	925 (18.5%)	2220 (99.8%)	2627 (87.6%)

5.1.1 Amazon reviews

This dataset categorizes Amazon reviews based on their scores, ranging from 1 to 5, into five clusters [72]. This study uses 5000 random samples from this dataset, with an equal number of records from each group. Figure 5 displays a histogram illustrating the frequency distribution of records in this dataset based on the number of tokens they contain. The horizontal axis in the histogram indicates the number of tokens in the dataset, while the vertical axis represents the number of records in the dataset for each token quantity.

On the other hand, Fig. 5 shows the red, black, and green vertical lines, indicating the maximum number of tokens that each model can process. In the histogram plot of the Amazon reviews dataset, the records located to the right of the red vertical line represent the texts that exceed the *max_seq_length* threshold for the RoBERTa model. Similarly, those situated to the right of the black dashed line represent texts that exceed the capacity of the ALBERT model. In addition, texts positioned to the right of the green dashed line indicate instances where the text length exceeds the limits of the BERT, DistilBERT, and MPNET models. The default version of these models can only consider text up to these limits and ignore text beyond this limit.

5.1.2 BBC news

This publicly available dataset comprises 2225 news articles from the BBC,[3] categorized into five topics: business, entertainment, politics, sport, and technology. Figure 6 shows the frequency distribution of token counts in the articles. This dataset has much longer texts than Amazon reviews, and almost all articles in the dataset exceed the *max_seq_length* limit of sentence transformer models. For instance, all news articles in this dataset contain more tokens than the *max_seq_length* threshold of the RoBERTa model.

5.1.3 Fake–real news

This dataset comprises real news articles from Reuters.com and fake news articles from Kaggle [73]. This study grouped 3000 records from this dataset with an equal number of real and fake news articles. Figure 7 presents histograms of article lengths in this dataset. Similar to the BBC News dataset, the majority of articles in this dataset exceed the maximum token limit of the sentence transformers.

5.1.4 Qualitative and quantitative analysis of datasets

Qualitative Analysis: Table 2 summarizes key statistics regarding the datasets, including the number of records, mean token counts, and the number and percentage of texts exceeding the *max_seq_length* thresholds for RoBERTa, ALBERT, and BERT/DistilBERT/MPNET models. For Amazon Reviews (a total of 5000 records with a mean of 81.53 tokens), 47.6% of all records exceed RoBERTa's limit, 32.1% exceed ALBERT's, and 18.5% surpass BERT/DistilBERT/MPNET's as illustrated in Fig. 5. For BBC News (a total of 2,225 records with a mean of 401.71 tokens), 100% of all records exceed RoBERTa's limit, 99.9% exceed ALBERT's, and 99.8% surpass BERT-/DistilBERT/MPNET's (Fig. 6). For Fake-Real News (a total of 3,000 records with the mean of 422.6 tokens), 96.1% of all records exceed RoBERTa's limit, 90.7% exceed ALBERT's, and 87.6% surpass BERT/DistilBERT/MPNET's (Fig. 7). These numbers show that most texts, especially in BBC News

[3] https://www.kaggle.com/competitions/learn-ai-bbc/data.

and Fake-Real News, are long, making these datasets suitable for testing the proposed SL and BL methods.

Qualitative Analysis: The Amazon Reviews dataset comprises user-generated reviews with scores from 1 to 5, reflecting different writing styles, lengths, and and levels of detail. Reviews vary from short, single-sentence opinions to longer narratives that include detailed product feedback, personal experiences, or comparisons with similar products. The informal tone, frequent use of slang, and occasional grammatical errors reflect the diverse backgrounds of the reviewers, adding complexity to clustering tasks. The BBC News dataset consists of news articles that are professionally written in a formal and well-structured language. The consistent use of precise grammar, standard formatting, and clear argumentation makes this dataset suitable for evaluating clustering methods that handle well-structured, long-form content. The Fake-True News dataset, combining real news articles from Reuters and fake news from Kaggle, presents a mix of text characteristics designed to inform, persuade, or mislead. True news articles exhibit professional writing with clear, concise language and correct grammar. In contrast, fake news articles often contain grammatical and linguistic errors, as well as repetitive or exaggerated language. The prevalence of such errors in fake news weakens the meaning and context of sentences.

5.2 Evaluation metrics

This study employs four primary metrics to evaluate clustering quality: Accuracy (ACC), Adjusted Rand Index (ARI), Normalized Mutual Information (NMI), and Silhouette Score. The first three metrics range from 0 to 1, with higher values indicating better alignment between predicted clusters and ground-truth labels. SIL ranges from -1 to 1, where values close to 1 indicate well-separated clusters, values close to 0 suggest overlapping clusters, and negative values imply incorrect cluster assignments. Together, these metrics provide a comprehensive view of clustering performance.

Additionally, the F1-Score, which is typically used in supervised learning, is also reported. While not standard in unsupervised settings, it can be applied when ground-truth labels are available, as in this study. However, because clustering algorithms produce unordered labels, an optimal label matching process is required before computing the F1-Score. This matching adds complexity and may reduce reliability, especially when a perfect alignment is not achievable. Therefore, the F1-Score is used here as a complementary metric, offering an additional perspective on how closely the predicted clusters resemble the ground-truth labels, rather than as a replacement for standard clustering metrics such as ARI, NMI, or SIL.

5.2.1 Accuracy (ACC)

ACC is the ratio of correctly predicted elements to total elements in a clustering process. This metric, which can be considered a simplified variant of the Rand Index. Consider a clustering problem with n examples:

g_i: the ground-truth cluster label of the ith example ($i = 1, 2, \ldots, n$)

p_i: The predicted cluster label of the ith example

Equation (9) defines ACC[74] as follows, where n denotes the total number of elements to be clustered:

$$\mathrm{ACC} = \frac{1}{n}\sum_{i=1}^{n}\delta\left(g_i, map\left(p_i\right)\right) \tag{9}$$

where $\delta\left(g_i, map\left(p_i\right)\right) = \begin{cases} 1 & if\ g_i = \mathrm{map}(p_i) \\ 0 & if\ g_i \neq m\ a\ p(p_i) \end{cases}$

5.2.2 Adjusted rand index (ARI)

ARI is a metric that measures the clustering performance by leveraging the randomness model. Equation (10) defines ARI [75]:

$$\mathrm{ARI} = \frac{\sum i,j\binom{n_{i,j}}{2} - \left[\sum i\binom{n_i}{2}\sum j\binom{n_j}{2}\right]/\binom{n}{2}}{\frac{1}{2}\left[\sum i\binom{n_i}{2}\right] - \left[\sum i\binom{n_i}{2}\sum j\binom{n_j}{2}\right]/\binom{n}{2}} \tag{10}$$

n is the total number of elements to be clustered, $n_(i,j)$ is the number of elements that appear both in p_i and g_j clusters. While p_i includes predicted values of ith cluster, g_j contains grand-truth values of jth cluster. n_i and n_j are the numbers of elements in p_i and g_j, respectively.

5.2.3 Normalised mutual information (NMI)

NMI is a metric derived from Mutual Information (MI), an entropy-based measure. MI's value can be affected by the number of clusters and elements within each cluster, leading to inconsistent results. NMI addresses this issue by scaling the MI value between 0 and 1. To compute NMI, let P represent the predicted clustering results and G be the ground-truth clustering results. The NMI calculation is given by Eq. (11) [76].

$$\mathrm{NMI}(P, G) = \frac{MI(P, G)}{\sqrt{H(P)H(G)}} \tag{11}$$

MI(P, G) denotes the MI value between cluster labels P and G, while $H(P)$ and $H(G)$ are entropies of P and G, respectively.

5.2.4 Silhouette score (SIL)

SIL is a robust metric to evaluate the quality of clustering by measures the balance between intra-cluster cohesion and inter-cluster separation for each data point. The score is defined as in Eq. (12):

$$\text{SIL} = \frac{1}{n}\sum_{i=1}^{n}\frac{b_i - a_i}{\max\{a_i, b_i\}} \tag{12}$$

where n is the total number of data points, a_i represents the avarage distance from the i-th data point to all other points in the same cluster, b_i is the smallest avarage distance from the ith data point to all points in any other cluster.

5.2.5 F-1 score

Although F1-score is a metric used in supervised learning as a harmonic mean of precision and recall, when ground-truth labels are available, it can be adapted for evaluating clustering algorithms. To compute the F1-score in a clustering context, predicted clusters must first be optimally aligned with ground-truth labels, typically using a mapping algorithm such as the Hungarian algorithm. This alignment ensures that each predicted cluster is paired with the most appropriate true label. The F1-score, ranging from 0 to 1, achieves its maximum value of 1 indicates precise and comprehensive cluster assignments.

5.3 Experimental results

In the experiments, we clustered the sentence embeddings of three datasets generated by BERT, RoBERTa, DistilBERT, ALBERT and MPNET-based sentence transformers using two different clustering methods: K-means and K-Medoids. The evaluations involved comparing the clustering performance of these five models in combination with SL, BL, and Default methods and investigating their compatibility. Both clustering methods were evaluated across 20 different random seeds for each experiment, with results averaged for consistency. The K-Means algorithm was configured with a maximum of 300 iterations, while the K-Medoids algorithm used the Euclidean distance metric. Across all tests, standard deviations were consistently low (often 0) indicating high stability. The ACC, ARI, NMI and SIL metrics also showed strong alignment, further supporting the reliability of the clustering results.

Additionally, we compared our SL and BL results with those of Longformer and BigBird, transformer models optimized for long text sequences using sparse attention mechanisms to reduce computational complexity and memory consumption and efficiently capture long-range dependencies. We generated embeddings for the three datasets using Longformer and BigBird and clustered them with K-Means and K-Medoids for benchmarking.

Table 3 presents the clustering performance of the K-means and K-medoids algorithms on the Amazon Reviews dataset. For K-means, the SL method consistently outperformed both the BL and Default methods across all SBERT models. The most successful pair was the BERT-SL configuration achieving an ACC of 0.716, an ARI of 0.125, an NMI of 0.148, and a SIL of 0217. All configurations of BERT and DistilBert, and the RoBERTa-SL pair surpassed Longformer and BigBird. Similar improvements were observed with K-Medoids. The BERT-SL configuration again demonstrated superior performance compared to its default counterpart, with ACC

Table 3 Clustering results of Amazon reviews

Clustering algorithm	Model	Method	ACC	ARI	NMI	SIL	F1-Score
K-means	BERT	Default	0.706	0.086	0.126	0.213	0.351
		SL	**0.716**	**0.125**	**0.148**	**0.217**	0.35
		BL	0.71	0.096	0.129	0.208	0.36
	RoBERTa	Default	0.66	0.01	0.014	0.334	0.24
		SL	**0.682**	**0.047**	**0.074**	**0.337**	0.283
		BL	0.661	0.01	0.014	0.34	0.242
	DistilBERT	Default	0.701	0.076	0.11	0.224	0.337
		SL	**0.705**	**0.084**	**0.121**	**0.229**	0.317
		BL	0.704	0.082	0.117	0.215	0.354
	ALBERT	Default	0.659	0.007	0.01	0.363	0.23
		SL	**0.673**	**0.009**	**0.012**	**0.373**	0.237
		BL	0.67	0.007	0.01	0.323	0.233
	MPNET	Default	0.656	0.007	0.009	0.393	0.221
		SL	**0.658**	**0.008**	**0.011**	**0.396**	0.233
		BL	0.655	0.007	0.01	0.395	0.219
	LongFormer	–	0.673	0.004	0.006	0.278	0.219
	BigBird	–	0.678	0.006	0.009	0.226	0.232
K-medoids	BERT	Default	0.703	0.087	0.112	0.183	0.316
		SL	**0.709**	**0.107**	**0.128**	**0.192**	0.325
		BL	0.7	0.106	0.122	0.186	0.33
	RoBERTa	Default	0.648	0.018	0.027	0.329	0.244
		SL	**0.663**	**0.022**	**0.037**	**0.332**	0.258
		BL	0.656	0.017	0.021	0.319	0.225
	DistilBERT	Default	0.7	0.084	0.11	0.187	0.327
		SL	**0.705**	**0.086**	**0.114**	**0.193**	0.323
		BL	0.701	0.085	0.103	0.183	0.313
	ALBERT	Default	0.649	0.006	0.008	0.347	0.208
		SL	**0.661**	**0.008**	**0.011**	**0.365**	0.224
		BL	0.66	0.007	0.009	0.341	0.23
	MPNET	Default	0.666	0.006	0.008	0.337	0.227
		SL	**0.669**	**0.007**	**0.009**	**0.353**	0.224
		BL	0.659	0.006	0.007	0.341	0.217
	LongFormer	–	0.665	0.006	0.007	0.25	0.224
	BigBird	–	0.668	0.007	0.011	0.278	0.234

The values that are highlighted in bold represent the highest scores obtained between the three methods (Default, SL and BL)

increasing from 0.703 to 0.709, ARI from 0.087 to 0.107, NMI from 0.112 to 0.128, and SIL from 0.316 to 0.325. While similar trends were observed across other models, all configurations of BERT and DistilBert, and the MPNET-SL pair showed higher performance than Longformer and BigBird.

Table 4 Clustering results of BBC News

Clustering Algorithm	Model	Method	ACC	ARI	NMI	SIL	F1-Score
K-means	BERT	Default	0.844	0.54	0.594	0.309	0.788
		SL	0.89	0.666	0.69	**0.373**	0.859
		BL	**0.905**	**0.708**	**0.723**	0.365	0.877
	RoBERTa	Default	0.954	0.856	0.828	0.406	0.938
		SL	0.958	0.863	0.835	**0.451**	0.936
		BL	**0.965**	**0.892**	**0.865**	0.442	0.954
	DistilBERT	Default	0.81	0.444	0.527	0.295	0.746
		SL	**0.903**	**0.7**	**0.714**	**0.376**	0.876
		BL	0.883	0.645	0.683	0.363	0.851
	ALBERT	Default	0.946	0.834	0.802	0.362	0.927
		SL	**0.948**	**0.839**	**0.81**	**0.417**	0.93
		BL	0.911	0.724	0.72	0.352	0.877
	MPNET	Default	0.956	0.875	0.849	0.386	0.955
		SL	0.962	0.882	0.851	**0.413**	0.949
		BL	**0.966**	**0.894**	**0.864**	0.405	0.955
	LongFormer	-	0.877	0.63	0.663	0.355	0.758
	BigBird	-	0.959	0.873	0.838	0.424	0.945
K-medoids	BERT	Default	0.831	0.491	0.543	0.291	0.741
		SL	0.887	0.657	0.695	**0.369**	0.855
		BL	**0.903**	**0.701**	**0.709**	0.356	0.871
	RoBERTa	Default	0.869	0.628	0.701	0.337	0.68
		SL	0.885	0.663	0.731	**0.376**	0.673
		BL	**0.886**	**0.666**	**0.74**	0.337	0.697
	DistilBERT	Default	0.794	0.375	0.435	0.261	0.623
		SL	**0.911**	**0.726**	**0.718**	**0.361**	0.881
		BL	0.855	0.555	0.574	0.298	0.756
	ALBERT	Default	0.885	0.652	0.699	0.304	0.78
		SL	**0.946**	**0.832**	**0.801**	**0.413**	0.926
		BL	0.91	0.72	0.713	0.344	0.875
	MPNET	Default	0.852	0.57	0.702	0.34	0.614
		SL	**0.89**	**0.677**	**0.742**	**0.349**	0.695
		BL	0.864	0.614	0.69	0.299	0.641
	LongFormer	-	0.862	0.617	0.645	0.341	0.81
	BigBird	-	0.956	0.863	0.831	0.42	0.941

The values that are highlighted in bold represent the highest scores obtained between the three methods (Default, SL and BL)

Despite lower baseline scores, RoBERTa, ALBERT, and MPNet benefited from the SL and BL enhancements. The BL method also generally produced better results than the Default method for all models. Overall, these results validate

the effectiveness of the SL and BL techniques in improving clustering performance on Amazon Review dataset. The greatest improvement was observed on the RoBERTa model, which achieved a 3% increase in ACC, rising from 0.660 with the Default method to 0.682 using the SL method. On the other hand, K-Means slightly outperformed K-Medoids in terms of clustering performance for all models, including Longformer and BigBird.

Table 4 presents the clustering results on the BBC News dataset. Both the SL and BL methods outperformed the Default configuration across most SBERT models under both clustering algorithms. The only exception was the ALBERT-SL configuration, which performed slightly worse than its Default counterpart. For K-Means, the SL method showed superiority to BL for DistilBERT and ALBERT in terms of ACC, ARI, and NMI, while BL yielded better results for BERT, RoBERTa, and MPNet. On the other hand, the SL method consistently achieved higher Silhouette Scores across all models, indicating better cluster cohesion. The MPNet-BL configuration achieved the highest overall performance, with an ACC of 0.966, ARI of 0.894, and NMI of 0.864. RoBERTa-BL also demonstrated comparably strong performance. All SBERT models with SL and BL configurations outperformed Longformer and were competitive with BigBird. For K-Medoids, SL and BL methods followed trends similar to those observed with K-Means. However, in contrast to K-Means, the SL method outperformed BL in the case of MPNet as well. The highest performance under K-Medoids was achieved by the ALBERT-SL configuration, with an ACC of 0.946, ARI of 0.832, NMI of 0.801, and a SIL of 0.413. All SBERT models delivered competitive results against both Longformer and BigBird.

Almost all models benefited from the SL and BL enhancements under both clustering methods on the BBC News dataset. The most notable improvement was observed with the DistilBERT model. For this model, SL led to an 11% improvement over the Default method with K-Means, and a 15% improvement with K-Medoids. Similar to the results of Amazon Reviews' dataset, the clustering performance of K-Medoids fell behind the K-Means.

Table 5 shows the clustering results on the Fake-True News dataset. Across both clustering algorithms and all models, the SL method consistently outperformed both the BL and Default configurations in almost all evaluations. The only exception was the RoBERTa-SL configuration, which performed slightly worse than its BL counterpart under the K-Medoids algorithm.Under K-Means, the best performing configuration was the BERT-SL pair, closely followed by DistilBERT-SL. While both configurations produced results competitive with Longformer, they still fell short of the performance achieved by BigBird. The remaining models, RoBERTa, ALBERT, and MPNET, demonstrated relatively low clustering performance. A similar pattern was observed under K-Medoids, where BERT-SL and DistilBERT-SL again emerged as the top performers. Although their results were comparable to Longformer, they could not match the performance of BigBird. The other models lagged significantly behind the benchmark models.

Despite the overall variability in model and method performance on the Fake-True News dataset, all models achieved improved clustering performance with the SL method compared to the Default configuration. Specifically, in K-Means

Table 5 Clustering results of Fake-Real news

Clustering algorithm	Model	Method	ACC	ARI	NMI	SIL	F1-Score
K-means	BERT	Default	0.749	0.499	0.398	0.297	0.853
		SL	**0.848**	**0.696**	**0.588**	**0.344**	0.917
		BL	0.758	0.516	0.417	0.307	0.859
	RoBERTa	Default	0.61	0.221	0.18	0.255	0.732
		SL	**0.639**	**0.2457**	**0.221**	**0.282**	0.729
		BL	0.624	0.248	0.201	0.266	0.651
	DistilBERT	Default	0.77	0.54	0.437	0.291	0.867
		SL	**0.839**	**0.679**	**0.57**	**0.344**	0.912
		BL	0.822	0.643	0.538	0.307	0.901
	ALBERT	Default	0.601	0.213	0.185	0.282	0.728
		SL	**0.637**	**0.275**	**0.226**	**0.292**	0.759
		BL	0.636	0.272	0.224	0.285	0.758
	MPNET	Default	0.579	0.159	0.129	0.247	0.695
		SL	**0.581**	**0.161**	**0.14**	**0.277**	0.693
		BL	0.562	0.123	0.105	0.256	0.668
	LongFormer	–	0.867	0.734	0.687	0.498	0.928
	BigBird	–	0.93	0.86	0.776	0.373	0.964
K-medoids	BERT	Default	0.737	0.474	0.376	0.295	0.844
		SL	**0.837**	**0.673**	**0.567**	**0.341**	0.91
		BL	0.727	0.454	0.359	0.299	0.837
	RoBERTa	Default	0.516	0.032	0.024	0.183	0.588
		SL	0.621	0.241	0.182	0.26	0.746
		BL	**0.673**	**0.346**	**0.292**	**0.197**	0.791
	DistilBERT	Default	0.757	0.514	0.413	0.287	0.859
		SL	**0.849**	**0.698**	**0.592**	**0.341**	0.918
		BL	0.792	0.584	0.485	0.301	0.882
	ALBERT	Default	0.634	0.269	0.215	0.243	0.758
		SL	**0.644**	**0.289**	**0.256**	**0.278**	0.835
		BL	0.609	0.218	0.181	0.283	0.73
	MPNET	Default	0.631	0.261	0.205	0.237	0.754
		SL	**0.632**	**0.263**	**0.206**	**0.263**	0.756
		BL	0.574	0.149	0.111	0.224	0.693
	LongFormer		0.87	0.74	0.686	0.497	0.93
	BigBird		0.926	0.852	0.765	0.372	0.962

The values that are highlighted in bold represent the highest scores obtained between the three methods (Default, SL and BL)

clustering, the BERT-SL and DistilBERT-SL configurations achieved 13% and 9% improvements in ACC, respectively over their Default counterparts. In K-Medoids clustering, BERT-SL and DistilBERT-SL showed gains of 14% and 12% respective over the Default method. Similar to other datasets, K-Means outperformed

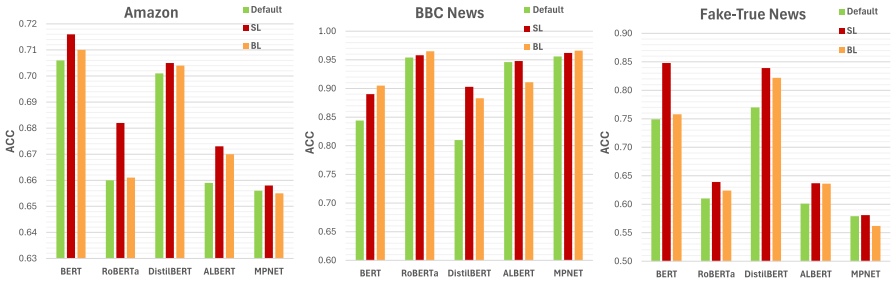

Fig. 8 ACC comparison of model&method combinations with K-means

K-Medoids, particularly in terms of the Silhouette Score, indicating better defined cluster structures.

As a result, out of the 30 tests conducted across three datasets, five models and two clustering algorithms, SL and BL achieved the highest scores in 24 and 6 cases, respectively. These results demonstrate that our proposed methods, especially SL, outperform the Default method in clustering long texts (Fig. 8). Although both algorithms produced largely parallel results, K-Means yielded slightly better performance compared to K-Medoids. Therefore, subsequent analyses were performed using K-Means.

To assess the scalability, efficiency, and computational resource requirements of the proposed SL and BL methods, we performed an additional analysis comparing them to the default configuration across all models using K-means clustering. Table 6 reports the runtime (in seconds), memory usage (in MB) and throughput (tokens processed per second) for 100, 1000 and 3000 records in the dataset, respectively. Due to space limitations, Table 6 only includes the results for the Fake-True News dataset, which has the largest volume. The corresponding results for Amazon Reviews and BBC News can be found in Appendix A and Appendix B respectively.

When assessing the results in terms of runtime, for instance on the BERT model, the Default method clustered 100 records in 13.9 s, 1000 records in 102.2 s, and 3000 records in 315.8 s. In constrast, the SL method on the same model required 90.3 s for 100 records, 822.9 s for 1000 records, and 2501.7 s for 3000 records. This represents an approximately eightfold increase in runtime relative to the Default method for 3000 records. This upward trend in runtime was consistently observed across all SBERT models when using SL. Such an increase is expected, as SL involves segmenting each text into individual sentences and performing additional aggregation operations. The BL method, less segmentation-intensive, took 50.2 s for 100 records, 471.9 s for 1000 records, and 1447.4 s for 3000 records, with reaching runtime scores between Default and SL. In addition, as expected, runtimes increased with the number of datasets, but the corresponding increase in throughput suggests that the efficiency of our methods improves with larger datasets. For example, the throughput values for BERT-SL configuration increased to 434, 481 and 485 as the number of records increased to 100, 1000 and 3000 respectively.

When compared to the benchmark models, SL demonstrated more efficient runtimes and effective throughput scores than Longformer and remained competitive

Table 6 Performance scores of all methods regarding scalability, efficiency for for fake–true news dataset

Model	Method	Sample size	Run-time (s)	Memory-usage (MB)	Throughput
BERT	Default	100	13.9	461	2818
		1000	102.2	484	3874
		3000	315.8	498	3840
	SL	100	90.3	495	434
		1000	822.9	656	481
		3000	2501.7	740	485
	BL	100	50.2	459	780
		1000	471.9	470	839
		3000	1447.4	478	838
RoBERTa	Default	100	9.1	467	4319
		1000	65	480	6095
		3000	193.3	495	6276
	SL	100	81.8	490	478
		1000	781.2	652	507
		3000	2413.5	740	503
	BL	100	46.7	460	838
		1000	435.1	473	910
		3000	1337.6	483	907
DistilBERT	Default	100	7.4	460	5267
		1000	56.4	483	7023
		3000	170.1	498	7132
	SL	100	46.6	492	839
		1000	422	656	938
		3000	1300.6	736	933
	BL	100	23.8	462	1646
		1000	240.1	470	1649
		3000	743.1	483	1632
ALBERT	Default	100	6.5	470	6059
		1000	55.6	486	7128
		3000	167.7	502	7235
	SL	100	54.8	481	714
		1000	525.1	493	754
		3000	1582.4	501	767
	BL	100	31.6	460	1240
		1000	282.9	466	1400
		3000	866.3	475	1400
MPNET	Default	100	14.3	465	2733
		1000	108.6	488	3647
		3000	317.8	495	3816
	SL	100	85.4	498	458
		1000	824.3	664	480
		3000	2495	752	486

Table 6 (continued)

Model	Method	Sample size	Run-time (s)	Memory-usage (MB)	Throughput
	BL	100	49.4	460	792
		1000	473.6	473	836
		3000	1458.5	483	832
LongFormer		100	119.7	448	327
		1000	1244.9	456	318
		3000	3673.4	460	330
BigBird		100	64.2	450	610
		1000	636	462	623
		3000	1941.5	476	625

with BigBird. Similar performance patterns were observed across all models in terms of the run-time impact of SL and BL. In particular, ALBERT and DistilBERT consistently exhibited shorter runtimes, highlighting their relative efficiency in terms of computational performance.

The memory usage of the Default, SL, and BL methods across all models shows different characteristics. BL and Default are more memory-efficient methods with excellent scalability (between 3 and 8% increase from 100 records to 3.000 records). SL has the highest memory usage and the least favorable scalability with approximately up to 50% increase, due to storing embeddings for each sentence separately and requiring extra space for aggregation process. SL's memory consumption also lagged behind the memory-efficient LongFormer and BigBird models with scalability rates of 3% and 6% respectively.

In addition, when evaluating the individual memory usage of the SBERT models, we observed that these models exhibit a highly similar memory consumption profile. The observed differences are primarily due to the design of the methods rather than the inherent architectures of the models.

6 Discussion and implications

The two proposed methods, particularly SL, consistently enhanced clustering performance across all models and datasets. The performance increases between the proposed and the Default methods are generally more noticeable in BBC News and Fake-True News than in Amazon. For instance, the most notable performance improvements were observed with K-Means: a 13% improvement for the BERT-SL combination over BERT-Default on the Fake-Real News dataset and a 12% improvement for the DistilBERT-SL configuration over DistilBERT-Default on the BBC News dataset. Compared to the Amazon Reviews dataset, which contains relatively short texts, the longer textual content in the BBC and Fake-True News datasets likely yielded these performance enhancements, confirming that SL excels in

handling longer texts. Another important finding is that, on the Amazon Reviews dataset, RoBERTa exhibited the greatest performance improvement with the SL method compared to its default counterpart, as clearly shown in Fig. 8. RoBERTa is also the model for which the Amazon Review texts most frequently exceed the *max_seq_length* limit as illustrated in Fig. 5 and Table 2. This validates that SL improves clustering performance more effectively as the number of texts over the *max_seq_length* limit increases.

Comparing the SL and BL, the possible reason for the better performance of SL over BL could be attributed to SL generating text embeddings sentence by sentence, while BL creates embeddings of texts containing a certain number of tokens. BL's approach may split sentences into different text blocks, leading to one part of the sentence being processed in the first block and the other in the next block. This may cause a potential meaning loss in the text. On the other hand, although the Default method omits some parts of the texts, the lack of significant performance differences with the proposed methods in some experiments may be because the initial parts of the texts often contain the general meaning of the entire text. In many cases, the later parts of long texts tend to elaborate on the ideas introduced in the first sentences.

To further demonstrate the superiority of the proposed SL and BL methods over the Default method in preserving semantics, we conducted an semantic analysis on the Fake-True News dataset by grouping texts into"Fake"and"True"categories based on their labels. For each group, we generated sentence embeddings using the Default, SL, and BL methods across five SBERT models. To quantify the semantic consistency of the embeddings, we computed the cosine similarity among all texts within the same group using sentence embeddings. This process involved calculating the pairwise cosine similarity for all texts in the group. The average of these values represents the overall similarity within a group. The results are presented in Table 7. A higher cosine similarity score indicates that the embeddings more effectively capture the shared semantic features of texts within the group.

Table 7 reveals that the proposed methods consistently produce sentence embeddings with higher cosine similarity, which is an indicator of intra-group cohesion, compared to the Default model, across all models and for both the Fake and True news groups. When comparing SL and BL, SL achieves the

Table 7 Average cosine similarities of the sentence embeddings vector generated by each model for each cluster

Cluster label	Cosine similarity					
	Method	BERT	RoBERTa	DistilBERT	ALBERT	MPNET
Fake	Default	0.912	0.949	0.965	0.921	0.978
	SL	**0.994**	0.958	**0.988**	0.973	**0.982**
	BL	0.971	**0.976**	0.977	**0.983**	0.956
True	Default	0.897	0.95	0.965	0.957	0.985
	SL	**0.995**	0.981	**0.99**	0.984	**0.99**
	BL	0.974	**0.987**	0.985	**0.987**	0.975

The values that are highlighted in bold represent the highest scores obtained between the three methods (Default, SL and BL)

highest cosine similarity for BERT, DistilBERT, and MPNET, whereas BL yields the highest scores for RoBERTa and ALBERT. However, despite BL producing slightly higher cosine similarity for RoBERTa and ALBERT, SL consistently achieves better overall clustering performance across all models as seen in Table 5. This is not a contradiction, as clustering performance should be evaluated not only based on intra-cluster cohesion (cosine similarity) but also on inter-cluster separation, which is captured by the Silhouette Score. For effective clustering, both high intra-cluster cohesion and high inter-cluster separation are essential. Therefore, the Silhouette Score offers a more comprehensive evaluation by considering both aspects. The higher Silhouette Scores achieved by SL in Table 5 further support its superior clustering performance compared to BL.

The results do not point out that any particular model is superior to others for clustering long texts. When evaluating the performance of individual models, BERT and DistilBERT demonstrated superior performance in the Amazon and Fake-Real News datasets while yielding the relatively low scores in BBC News. Conversely, MPNET achieved the highest score in the BBC News dataset but displayed inconsistent performance, lagging significantly behind in the other two datasets. Unlike the other models, which were trained on the same corpora with BERT using different methodologies, MPNET was pretrained on a distinct 160GB corpus [67]. It appears that the textual characteristics of the BBC News dataset align more closely with the corpus used to pretrain MPNET. These observations suggest that the effectiveness of sentence embedding models may be influenced by the nature and content of their pre-training corpora.

Although the results demonstrate that generic SBERT models can be applied to text clustering, it is proposed that fine-tuning these models in a supervised manner with relevant datasets could potentially enhance their performance. Furthermore, the BBC News dataset, contains texts that are more well-structured in terms of grammar, semantics, and coherence compared to the others, appears to enable SBERT models to achieve higher clustering performance. In contrast, a high frequency of grammatical and syntactic errors, slang, and repetitive or exaggerated language use in the Amazon Reviews and Fake-True News datasets, may have hindered the SBERT models' ability to produce effective sentence embeddings, potentially leading to reduced clustering performance.

Another finding from Fig. 8 is that BERT and DistilBERT models exhibit better compatibility with SL and BL than others. This is evident in the substantial differences between the results of BERT and DistilBERT when using SL and BL versus Default. In contrast, the other models do not consistently performs such substantial improvement across methods. Notably, in all BERT and DistilBERT experiments, SL consistently outperforms both BL and the Default. Figure 9 visualizes the sentence embeddings generated by BERT for the Default, SL, and BL methods, projected into a two-dimensional space using UMAP. The points are colored based on their ground-truth cluster labels. This visualization illustrates that BERT generates more distinct text clusters with SL and BL compared to Default. It also provides insights regarding how Amazon sentence embeddings are overlapped and why their clustering results are lower than other datasets.

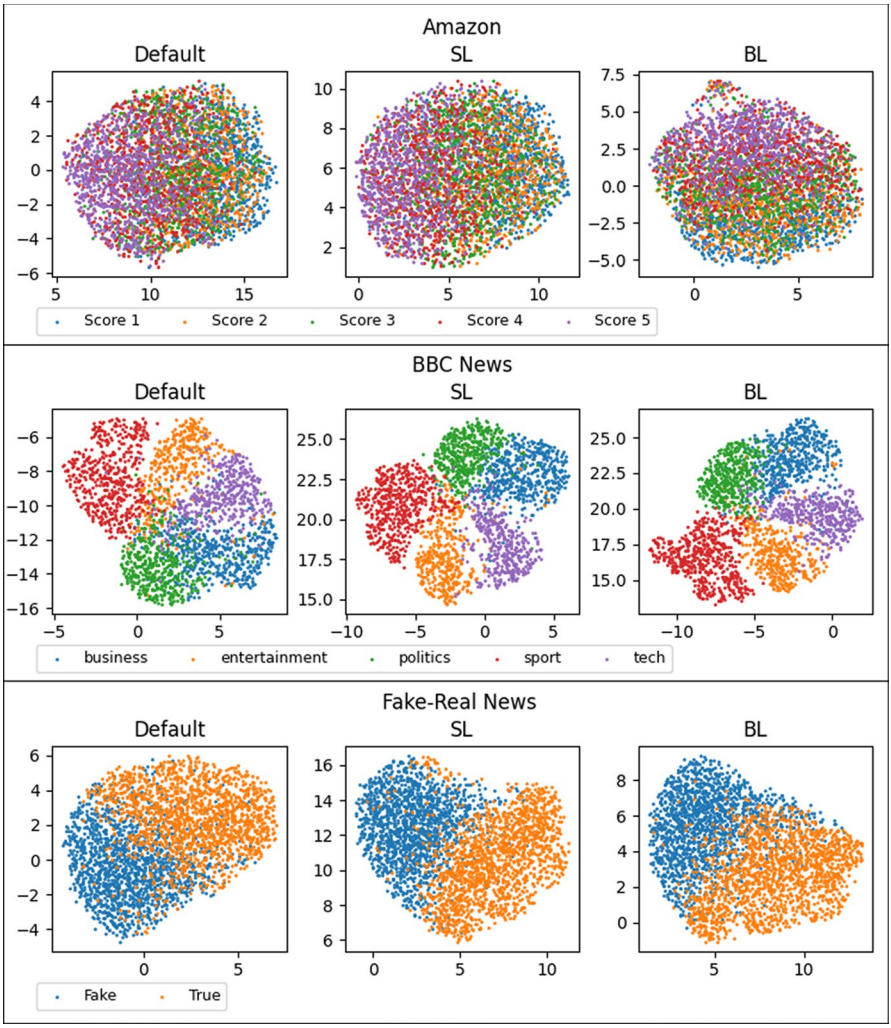

Fig. 9 Visualizations of BERT's text embeddings with three methods after UMAP dimension reduction

The heatmap in Fig. 10 presents the correlations among all SBERT models based on ACC, ARI, NMI, and SIL metrics, evaluated across the three methods and datasets used in this study. It indicates a strong correlation between BERT and DistilBERT, while RoBERTa, MPNET, and ALBERT also exhibit a high correlation. Consequently, models with high correlation can be considered viable substitutes for one another. In particular, the advantage of DistilBERT over BERT regarding size and speed increases its preference.

The memory consumption and runtime characteristics of the Default, SL, and BL methods across all models highlight clear trade-offs between efficiency and effectiveness. While the Default method demonstrates the lowest runtime,

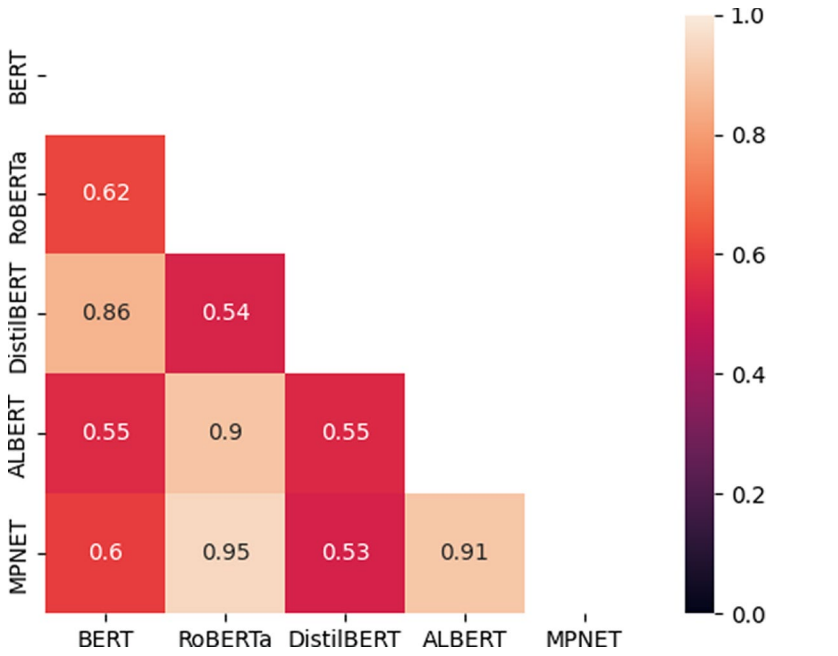

Fig. 10 Correlation heatmap of models

it also yields the poorest clustering success. In contrast, the SL method significantly increases runtime compared to Default but delivers the best clustering results. When benchmarked against LongFormer and BigBird, SL's runtime can still be considered within a reasonable range. Meanwhile, the BL method consistently maintains a stable runtime, positioning itself between Default and SL. In terms of memory usage, the SL method emerges as the most memory-intensive approach, exhibiting substantially higher memory usage compared to the BL and Default methods, which demonstrate scalable memory performance as dataset size increases. In this regard, LongFormer and BigBird outperform SL by exhibiting more scalable memory efficiency. The only exception to this pattern is the ALBERT model. When SL is applied to ALBERT, it displays notably scalable memory usage, unlike the trend observed in other models. This finding validates that ALBERT's architecture, which share the same parameters across layers, is designed to reduce memory consumption.

In light of the results and findings of this study, the following practical guidelines may assist practitioners:

- K-means is recommended for clustering long texts when using SL and BL methods, as it consistently outperforms K-medoids in terms of accuracy and cluster separation across all datasets.

- Although SL has a higher memory footprint and runtime than BL, its superior clustering performance on long texts makes it the more advantageous option. However, in resource-constrained environments, BL may be a preferable alternative to the Default method.
- For SBERT model selection, BERT and DistilBERT are recommended due to their strong compatibility with the SL method. Among them, DistilBERT is particularly advantageous given its smaller size and faster processing time.
- It should be noted that SBERT models perform more effectively on texts that are grammatically and syntactically well-formed and written in a coherent, narrative style. In contrast, texts with frequent spelling errors, grammatical issues, or heavy use of slang may hinder the models' ability to produce high-quality embeddings, potentially leading to suboptimal performance.

7 Limitations and future directions

While the proposed SL and BL methods demonstrate significant improvements in clustering performance for long texts, they are subject to several limitations:

First, the clustering algorithms used in this study, K-means and K-medoids, require a predefined number of clusters (k). However, in real-world scenarios, k is often unknown, making clustering more challenging. As our datasets include ground-truth labels, we focused on extracting effective embeddings for long texts. Future work could address this limitation by exploring clustering methods that automatically infer the optimal number of clusters.

Second, neither SL nor BL include advanced mechanisms to identify and emphasize the most semantically important parts of long texts. For example, SL generates the embeddings of all sentences individually and averages them, implicitly treating all sentences as equally important. However, in many cases, some sentences carry more semantic weight than others. Ignoring this variation may dilute the influence of key sentences, potentially resulting in suboptimal document embeddings. Future work could address these limitations by incorporating a weighted averaging mechanism, where sentence importance is determined, such as through attention weights from transformer models, and embeddings are aggregated accordingly. Alternatively, integrating text summarization techniques that identify key sentences or sections into the embedding pipeline could enhance both SL and BL by focusing on the most informative parts of the text.

Third, the performance of the proposed approach varies depending on the dataset characteristics, which may limit its generalizability. Datasets with well-structured texts yield higher clustering performance while those with informal language, slang, or grammatical errors yield lower performance. This variability suggests that the proposed methods may not perform equally well across all text types, particularly in noisy, user-generated content such as social media posts. Future research could focus on improving the robustness of these methods to handle different text

qualities, possibly through pre-processing techniques to reduce noise or by incorporating error-tolerant embedding strategies.

8 Conclusions

SBERT-based LLMs are state-of-the-art methods that have shown remarkable success in generating independent sentence embeddings in recent years. However, these models have maximum token constraints making them unsuitable for certain text-processing tasks when dealing with texts exceeding these limits. One such task is text clustering, and this study focuses on clustering long texts with SBERT models. To overcome their limitations, we presented two novel methods, SL and BL, to generate long-text sentence embeddings. SL divides the text into sentences, and BL divides the text into blocks containing words up to the maximum token limit.

The experimental results with two clustering algorithms and five SBERT models on three long-text datasets indicate that the proposed methods, especially SL, are superior to the existing Default method in almost all cases of 30 tests. The results suggest that the aggregation mechanism in the SL and BL better preserves long-range semantic dependencies in long texts, mitigating the information loss caused by Default's truncation approach. Regarding efficiency, SL requires more time than other methods but produces results within reasonable durations compared to benchmark models. In terms of scalability, SL yields weaker scores compared to both BL and Default methods as well as benchmark models.

Evaluating models individually, none of the models clearly outclassed the others in clustering long texts. However, BERT and DistilBERT showed better harmony with SL and BL. In addition, the model correlation analysis showed potential model interchangeability. Accordingly, BERT and DistilBERT, which produce parallel results, can be used for each other; similarly, RoBERTa, MPNET, and relatively ALBERT can be substituted. In the comparison of clustering algorithms, K-means demonstrates a slight preference over K-medoids.

In conclusion, this study provides valuable insights into the effective clustering of long texts using SBERT-based generic models through its findings, implications, practical guidance for practitioners and future research directions.

The source code of SL and BL methods is shared on the GitHub repo: https://github.com/yasinor/SBERT_Long_Text_Clustering.git.

The scores for the Amazon reviews dataset

See Table 8.

Table 8 Performance scores of all methods regarding scalability, efficiency for the Amazon Reviews dataset

Model	Method	Sample size	Run-time (s)	Memory-usage (MB)	Throughput
BERT	Default	100	13.0	468	587
		1000	79.8	485	970
		5000	374.4	588	1007
	SL	100	27.7	483	275
		1000	151.6	578	511
		5000	751.4	735	502
	BL	100	21.5	449	353
		1000	155.7	457	497
		5000	773.0	534	488
RoBERTa	Default	100	11.3	465	673
		1000	59.1	480	1311
		5000	277.3	574	1359
	SL	100	20.9	485	364
		1000	148.9	575	520
		5000	712.7	708	529
	BL	100	17.1	449	447
		1000	122.3	459	633
		5000	595.8	752	633
DistilBERT	Default	100	8.2	464	931
		1000	39.6	479	1954
		5000	204.2	577	1846
	SL	100	11.9	480	637
		1000	81.3	573	953
		5000	377.5	739	998
	BL	100	12.8	451	597
		1000	80.1	453	966
		5000	401.3	533	939
ALBERT	Default	100	7.1	467	1079
		1000	45.3	486	1711
		5000	212.9	569	1771
	SL	100	12.7	461	599
		1000	98.6	469	785
		5000	458.2	551	822
	BL	100	12.4	450	614
		1000	91.7	458	844
		5000	453.2	538	832
MPNET	Default	100	13.1	466	581
		1000	78.6	487	985
		5000	378.4	578	996
	SL	100	22.1	486	344
		1000	149.8	575	517

Table 8 (continued)

Model	Method	Sample size	Run-time (s)	Memory-usage (MB)	Throughput
		5000	713.8	728	528
	BL	100	21.4	448	356
		1000	152.5	457	508
		5000	812.8	545	464
LongFormer		100	100.2	443	76
		1000	908.2	454	85
		5000	4866.9	528	77
BigBird		100	12.8	441	597
		1000	111.0	450	697
		5000	541.4	524	696

The scores for the BBC News dataset

See Table 9.

Table 9 Performance scores of all methods regarding scalability, efficiency for the BBC News dataset

Model	Method	Sample Size	Run-time (s)	Memory-usage (MB)	Throughput
BERT	Default	100	12.2	460	2801
		1000	100.4	482	3623
		2250	224.8	487	3744
	SL	100	61.2	492	559
		1000	613.9	628	592
		2250	1394.6	715	603
	BL	100	43.0	458	795
		1000	431.2	476	844
		2250	977.8	488	861
RoBERTa	Default	100	8.9	468	3848
		1000	59.5	484	6111
		2250	138.4	488	6080
	SL	100	60.5	494	566
		1000	586.9	636	620
		2250	1352.0	711	622
	BL	100	37.5	462	911
		1000	403.6	476	901
		2250	906.5	492	928
DistilBERT	Default	100	7.2	460	4731
		1000	53.7	479	6774
		2250	125.1	490	6728
	SL	100	29.3	492	1169
		1000	309.5	624	1175
		2250	714.2	717	1178
	BL	100	25.0	461	1367
		1000	218.5	476	1664
		2250	498.3	491	1689
ALBERT	Default	100	6.4	470	5306
		1000	53.1	489	6851
		2250	120.3	504	6994
	SL	100	39.2	479	872
		1000	377.7	494	963
		2250	874.2	502	963
	BL	100	23.9	458	1429
		1000	252.0	473	1443
		2250	573.5	486	1467
MPNET	Default	100	16.2	463	2108
		1000	105.7	486	3442
		2250	235.1	492	3580
	SL	100	60.6	496	564
		1000	605.0	627	601
		2250	1394.0	722	604

Table 9 (continued)

Model	Method	Sample Size	Run-time (s)	Memory-usage (MB)	Throughput
	BL	100	44.7	460	765
		1000	429.1	473	848
		2250	989.0	492	851
LongFormer		100	108.7	450	315
		1000	1105.6	463	329
		2250	2602.3	480	323
BigBird		100	51.9	449	659
		1000	535.4	461	679
		2250	1379.1	477	610

Author Contributions Yasin Ortakci developed the methodology and coded the necessary software. He also wrote the manuscript text and prepared the figures pertaining to the analysis and visualization of the results. Burak Borhan contributed to the conduct, analysis and visualization of the experimental tests.

Funding Open access funding provided by the Scientific and Technological Research Council of Türkiye (TÜBİTAK).

Data availability The data used in this paper are all from publicly available datasets.

Declarations

Conflict of interest The author states that there are no competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Ethics approval This study did not involve human or animal subjects, and thus, no ethical approval was required.

References

1. Al-Anazi S, AlMahmoud H, Al-Turaiki I (2016) Finding similar documents using different clustering techniques. Procedia Comput Sci 82:28–34
2. Zhang Z, Fang M, Chen L, Namazi-Rad M-R (2022) Is neural topic modelling better than clustering? An empirical study on clustering with contextual embeddings for topics. arXiv:2204.09874
3. Kamalloo E, Zhang X, Ogundepo O, Thakur N, Alfonso-Hermelo D, Rezagholizadeh M, Lin J (2023) Evaluating embedding APIs for information retrieval. arXiv:2305.06300

4. Xie X, Wang B (2018) Web page recommendation via twofold clustering: considering user behavior and topic relation. Neural Comput Appl 29:235–243
5. Ghadimi A, Beigy H (2023) SGCSumm: an extractive multi-document summarization method based on pre-trained language model, submodularity, and graph convolutional neural networks. Expert Syst Appl 215:119308
6. Huang L, Liu G, Chen T, Yuan H, Shi P, Miao Y (2021) Similarity-based emergency event detection in social media. J Saf Sci Resil 2(1):11–19
7. Moura A, Lima P, Mendonça F, Mostafa SS, Morgado-Dias F (2023) On the use of transformer-based models for intent detection using clustering algorithms. Appl Sci 13(8):5178
8. Schluter N (2018) The word analogy testing caveat. In: Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies: Volume 2 (Short Papers). Association for Computational Linguistics, pp 242–246
9. Yin B, Zhao M, Guo L, Qiao L (2023) Sentence-BERT and k-means based clustering technology for scientific and technical literature. In: 2023 15th International Conference on Computer Research and Development (ICCRD). IEEE, pp 15–20
10. Pappagari R, Zelasko P, Villalba J, Carmiel Y, Dehak N (2019) Hierarchical transformers for long document classification. In: 2019 IEEE automatic speech recognition and understanding workshop (ASRU). IEEE, pp 838–844
11. Selva Birunda S, Kanniga Devi R (2021) A review on word embedding techniques for text classification. In: Innovative data communication technologies and application: proceedings of ICIDCA 2020. Springer, pp 267–281
12. Soumya George K, Joseph S (2014) Text classification by augmenting bag of words (BOW) representation with co-occurrence feature. IOSR J Comput Eng 16(1):34–38
13. Mendonça I, Trouvé A, Fukuda A, Murakami KJ, Tsai CF, Hu YH, Wang MC, Liu KE, Yu X, Yuan Y et al (2019) On clustering algorithms: applications in word-embedding documents. J Comput 14(2):88–92
14. Singh AK, Shashi M (2019) Vectorization of text documents for identifying unifiable news articles. Int J Adv Comput Sci Appl 10(7)
15. Sundararaman D, Srinivasan S (2017) Twigraph: discovering and visualizing influential words between Twitter profiles. In: Social Informatics: 9th International Conference, SocInfo 2017, Oxford, UK, September 13–15, 2017, Proceedings, Part II 9. Springer, pp 329–346
16. Tao J, Jia L, Wan MC, Meng JH (2020) The Text modeling method of Tibetan text combining Word2vec and improved TF-IDF. J Phys Conf Ser 1601(4):042007
17. Lilleberg J, Zhu Y, Zhang Y (2015) Support vector machines and word2vec for text classification with semantic features. In: 2015 IEEE 14th International Conference on Cognitive Informatics and Cognitive Computing (ICCI* CC). IEEE, pp 136–140
18. Omar A (2020) Feature selection in text clustering applications of literary texts: a hybrid of term weighting methods. Int J Adv Comput Sci Appl 11(2)
19. Bharti KK, Singh PK (2015) Hybrid dimension reduction by integrating feature selection with feature extraction method for text lustering. Expert Syst Appl 42(6):3105–3114
20. Mikolov T, Chen K, Corrado G, Dean J (2013) Efficient estimation of word representations in vector space. arXiv:1301.3781
21. Mikolov T, Sutskever I, Chen K, Corrado GS, Dean J (2013) Distributed representations of words and phrases and their compositionality. In: Advances in neural information processing systems, 26
22. Yu C, Zhao Y (2022) Mining of risk perception dimensions of Chinese tourists' outbound tourism based on word vector method. Front Psychol 13:1091065
23. Baek J-W, Chung K-Y (2021) Multimedia recommendation using Word2Vec-based social relationship mining. Multimedia Tools Appl 80:34499–34515
24. Pennington J, Socher R, Manning CD (2014) Glove: global vectors for word representation. In: Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp 1532–1543
25. Haagsma H, Bjerva J (2016) Detecting novel metaphor using selectional preference information. In: Proceedings of the fourth workshop on metaphor in NLP, pp 10–17
26. Jo H, Cinarel C (2019) Delta-training: Simple semi-supervised text classification using pretrained word embeddings. arXiv:1901.07651
27. Bojanowski P, Grave E, Joulin A, Mikolov T (2017) Enriching word vectors with subword information. Trans Assoc Comput Linguist 5:135–146

28. Ghozali I, Sungkono KR, Sarno R, Abdullah R (2023) Synonym based feature expansion for Indonesian hate speech detection. Int J Electr Comput Eng IJECE 13(1):1105
29. Umer M, Imtiaz Z, Ahmad M, Nappi M, Medaglia C, Choi GS, Mehmood A (2023) Impact of convolutional neural network and FastText embedding on text classification. Multimedia Tools Appl 82(4):5569–5585
30. Alhoshan W, Ferrari A, Zhao L (2023) Zero-shot learning for requirements classification: An exploratory study. Inf Softw Technol 159:107202
31. Howard J, Ruder S (2018) Universal language model fine-tuning for text classification. arXiv:1801.06146
32. Sarzynska-Wawer J, Wawer A, Pawlak A, Szymanowska J, Stefaniak I, Jarkiewicz M, Okruszek L (2021) Detecting formal thought disorder by deep contextualized word representations. Psychiatry Res 304:114135
33. Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser Ł, Polosukhin I (2017) Attention is all you need. In: Advances in neural information processing systems, 30
34. Brown T, Mann B, Ryder N, Subbiah M, Kaplan JD, Dhariwal P, Neelakantan A, Shyam P, Sastry G, Askell A et al (2020) Language models are few-shot learners. Adv Neural Inf Process Syst 33:1877–1901
35. Devlin J, Chang M-W, Lee K, Toutanova K (2018) Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv:1810.04805
36. Ethayarajh K (2019) How contextual are contextualized word representations? Comparing the geometry of BERT, ELMo, and GPT-2 embeddings. arXiv:1909.00512
37. Muennighoff N (2022) Sgpt: Gpt sentence embeddings for semantic earch. arXiv:2202.08904
38. Subakti A, Murfi H, Hariadi N (2022) The performance of BERT as data representation of text clustering. J Big Data 9(1):1–21
39. Moura A, Lima P, Mendonça F, Mostafa SS, Morgado-Dias F (2023) On the use of transformer-based models for intent detection using clustering algorithms. Appl Sci 13(8):5178
40. Mehta V, Bawa S, Singh J (2021) WEClustering: word embeddings based text clustering technique for large datasets. Complex Intell Syst 7:3211–3224
41. Hosseini S, Varzaneh ZA (2022) Deep text clustering using stacked AutoEncoder. Multimedia Tools Appl 81(8):10861–10881
42. Li Y, Cai J, Wang J (2020) A text document clustering method based on weighted Bert model. In: 2020 IEEE 4th Information Technology, Networking, Electronic and Automation Control Conference (ITNEC), vol 1. IEEE, pp 1426–1430
43. Reimers N, Gurevych I (2019) Sentence-bert: sentence embeddings using siamese bert-networks. arXiv:1908.10084
44. Sorana C, Luc D, Julien B et al (2023) Application and evaluation of sentence embedding and clustering methods in the context of concept hierarchy construction. Procedia Comput Sci 225:3479–3487
45. Abdalgader K, Matroud AA, Hossin K (2024) Experimental study on short-text clustering using transformer-based semantic similarity measure. PeerJ Comput Sci 10:e2078
46. Kiran N, Ragha L, Ghorpade T (2023) Summarizing students' text-only answer sheet using SBERT and K-means clustering and evaluating it using semantic search. In: International Conference on Artificial Intelligence and Knowledge Processing, pp 310–323
47. Ortakci Y (2024) Revolutionary text clustering: investigating transfer learning capacity of SBERT models through pooling techniques. Int J Eng Sci Technol 55:101730
48. Hammami E, Faiz R (2022) Text clustering based on multi-view representations. In: CIRCLE'22: Conference of the Information Retrieval Communities in Europe, pp 3178
49. Conceição L, Rodrigues V, Meira J, Marreiros G, Novais P (2022) Supporting argumentation dialogues in group decision support systems: an approach based on dynamic clustering. Appl Sci 12(21):10893
50. Chu Y, Cao H, Diao Y, Lin H (2023) Refined sbert: representing sentence bert in manifold space. Neurocomputing 555:126453
51. Gündoğan E, Kaya M (2023) An Article Similarity-Based Approach for Planning Conference Sessions. In: 2023 13th International Conference on Advanced Computer Information Technologies (ACIT). IEEE, pp 591–594
52. Yin B, Zhao M, Guo L, Qiao L (2023) Sentence-BERT and k-means based clustering technology for scientific and technical literature. In: 2023 15th International Conference on Computer Research and Development (ICCRD). IEEE, pp 15–20

53. Christou L, Bompotas A, Makris C (2024) Document embeddings for long texts from transformers and autoencoders
54. Guedes GB, da Silva AEA (2024) Classification and clustering of sentence-level embeddings of scientific articles generated by contrastive learning. arXiv:2404.00224
55. Wang Z, Zhu Y, Li Y, Qiang J, Yuan Y, Zhang C (2024) Asymmetric short-text clustering via prompt. N Gener Comput 42(4):599–615
56. Beltagy I, Peters ME, Cohan A (2020) Longformer: the long-document transformer. arXiv:2004.05150
57. Zaheer M, Guruganesh G, Dubey KA, Ainslie J, Alberti C, Ontanon S, Pham P, Ravula A, Wang Q, Yang L et al (2020) Big bird: Transformers for longer sequences. J Adv Neural Inf Process Syst 33:17283–17297
58. Zhu Y, Kiros R, Zemel R, Salakhutdinov R, Urtasun R, Torralba A, Fidler S (2015) Aligning books and movies: Towards story-like visual explanations by watching movies and reading books. In: Proceedings of the IEEE International Conference on Computer Vision, pp 19–27
59. d'Sa AG, Illina I, Fohr D (2020) Bert and fasttext embeddings for automatic detection of toxic speech. In: 2020 International Multi-Conference on:"Organization of Knowledge and Advanced Technologies"(OCTA), pp 1–5. IEEE
60. Ye, Zhihao and Jiang, Gongyao and Liu, Ye and Li, Zhiyong and Yuan, Jin (2020) Document and word representations generated by graph convolutional network and bert for short text classification. In: 24th European Conference on Artificial Intelligence (ECAI 2020), pp 2275–2281
61. Wu Y, Schuster M, Chen Z, Le QV, Norouzi M, Macherey W, Krikun M, Cao Y, Gao Q, Macherey K et al (2016) Google's neural machine translation system: Bridging the gap between human and machine translation. arXiv:1609.08144
62. Liu Y, Ott M, Goyal N, Du J, Joshi M, Chen D, Levy O, Lewis M, Zettlemoyer L, Stoyanov V (2019) Roberta: a robustly optimized bert pretraining approach. arXiv:1907.11692
63. Sennrich R, Haddow B, Birch A (2015) Neural machine translation of rare words with subword units. arxiv:1508.0790
64. Thompson L, Mimno D(2020) Topic modeling with contextualized word representation clusters. arXiv:2010.12626
65. Lan Z, Chen M, Goodman S, Gimpel K, Sharma P, Soricut R (2019) Albert: a lite bert for self-supervised learning of language representations. arXiv:1909.11942
66. Sanh V, Debut L, Chaumond J, Wolf T (2019) DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv:1910.01108
67. Song K, Tan X, Qin T, Lu J, Liu T-Y (2020) Mpnet: masked and permuted pre-training for language understanding. Adv Neural Inf Process Syst 33:16857–16867
68. Cer D, Yang Y, Kong S-y, Hua N, Limtiaco N, John RS, Constant N, Guajardo-Cespedes M, Yuan S, Tar C et al (2018) Universal sentence encoder. arXiv:1803.11175
69. Conneau A, Kiela D, Schwenk H, Barrault L, Bordes A (2017) Supervised learning of universal sentence representations from natural language inference data. arXiv:1705.02364
70. Jain AK (2010) Data clustering: 50 years beyond K-means. Pattern Recogn Lett 31(8):651–666
71. Rdusseeun LKPJ, Kaufman P (1987) Clustering by means of medoids. In: Proceedings of the Statistical Data Analysis Based on the L1 Norm Conference, Neuchatel, Switzerland, pp 31–28
72. Zhang X, Zhao J, LeCun Y (2015) Character-level convolutional networks for text classification. In: Advances in neural information processing systems, 28
73. Ahmed H, Traore I, Saad S (2018) Detecting opinion spams and fake news using text classification. Secur Privacy 1(1):e9
74. Xu J, Xu B, Wang P, Zheng S, Tian G, Zhao J (2017) Self-taught convolutional neural networks for short text clustering. Neural Netw 88:22–31
75. Janani R, Vijayarani S (2019) Text document clustering using spectral clustering algorithm with particle swarm optimization. Expert Syst Appl 134:192–200
76. Guan R, Zhang H, Liang Y, Giunchiglia F, Huang L, Feng X (2020) Deep feature-based text clustering and its explanation. IEEE Trans Knowl Data Eng 34(8):3669–3680